

داده کاوی

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل پنجم: Pattern mining: advanced methods

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو و امیرحسین همتی

فهرست

- 5.1 Mining various kinds of patterns
 - 5.1.1 Mining multilevel associations
 - 5.1.2 Mining multidimensional associations
 - 5.1.3 Mining quantitative association rules
 - 5.1.4 Mining high-dimensional data
 - 5.1.5 Mining rare patterns and negative patterns
- 5.2 Mining compressed or approximate patterns
 - 5.2.1 Mining compressed patterns by pattern clustering
 - 5.2.2 Extracting redundancy-aware top-k patterns
- 5.3 Constraint-based pattern mining
 - 5.3.1 Pruning pattern space with pattern pruning constraints
 - 5.3.2 Pruning data space with data pruning constraints
 - 5.3.3 Mining space pruning with succinctness constraints
- 5.4 Mining sequential patterns
 - 5.4.1 Sequential pattern mining: concepts and primitives
 - 5.4.2 Scalable methods for mining sequential patterns
 - 5.4.3 Constraint-based mining of sequential patterns
- 5.5 Mining subgraph patterns
 - 5.5.1 Methods for mining frequent subgraphs
 - 5.5.2 Mining variant and constrained substructure patterns
- 5.6 Pattern mining: application examples
 - 5.6.1 Phrase mining in massive text data
 - 5.6.2 Mining copy and paste bugs in software programs

Section 5.1

5.1.1. Mining multilevel associations

کاوش ارتباطات چندسطحی

کاوش ارتباطات چندسطحی (Multilevel Association Mining) به کشف الگوها و روابط در سطوح مختلف گرانولاریته در یک مجموعه داده می‌پردازد. این تکنیک امکان بررسی وابستگی‌ها را نه تنها در سطح آیتم‌های خاص (مانند «شیر کاکائو»)، بلکه در سطوح بالاتر انتزاع – مانند دسته‌بندی‌ها («نوشیدنی‌ها»)، سلسله مراتب («غذا → لبنیات → شیر») یا ترکیب ویژگی‌ها – فراهم

می‌کند. این رویکرد بینش‌های عمیق‌تری نسبت به رفتار کاربران ارائه می‌دهد و در سناریوهایی مانند تحلیل فروش، بازاریابی و طراحی محصول بسیار مفید است.

مفاهیم کلیدی

- قواعد ارتباطی چندسطحی (Multilevel Association Rules):** این قواعد مفهوم سنتی قواعد ارتباطی را به چندین سطح انتزاع گسترش می‌دهند. به جای تمرکز فقط بر روی آیتم‌های خام (مانند «نان ساندویچی»), قواعد چندسطحی می‌توانند روابطی مانند «اگر مشتری لبنیات بخرد، احتمالاً نوشیدنی‌های انرژی‌زا می‌خرد» را کشف کنند. این امر درک جامع‌تری از داده و رفتار مشتری ارائه می‌دهد.
- ساختارهای سلسله‌مراتبی (Hierarchical Structures):** کاوش ارتباطات چندسطحی اغلب بهره از ساختارهای سلسله‌مراتبی یا طبقه‌بندی‌های موجود در داده می‌برد. مثلاً در یک مجموعه داده خرده‌فروشی، آیتم‌ها ممکن است در سطوحی مانند **دسته** → **زیردسته** → **برند** → **محصول** سازماندهی شوند. کاوش در این سطوح امکان کشف الگوهای کلی‌تر (مثلاً در سطح دسته) یا جزئی‌تر (در سطح محصول) را فراهم می‌کند.

مثالی از کاوش ارتباطات چندسطحی در پایتون

بیایید از یک مجموعه داده خرده‌فروشی با ساختار سلسله‌مراتبی (مانند دسته، زیردسته و محصول) استفاده کنیم تا نحوه کاوش ارتباطات چندسطحی را در پایتون نشان دهیم:

```
In [1]: # Install the mlxtend library if not already installed
# pip install mlxtend

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (hierarchy) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده از یک مجموعه داده خرده‌فروشی با سلسله‌مراتب دسته‌بندی
# تعلق داره ('Fruit' مثلاً) هر آیتم به یک دسته
# استفاده می‌شه **Multilevel Association Rules** این ساختار برای شبیه‌سازی **قوانین انجمنی چندسطحی
# رو بررسی کنیم ('Fruit' → 'Fruit' مثل) و هم بین دسته‌ها ('Apple' → 'Banana' مثل) در این نوع تحلیل، می‌تونیم هم روابط بین آیتم‌های خاص
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['Apple', 'Banana', 'Apple', 'Banana', 'Apple', 'Banana', 'Orange', 'Apple', 'Banana', 'Orange'],
    'Category': ['Fruit', 'Fruit', 'Fruit', 'Fruit', 'Fruit', 'Fruit', 'Fruit', 'Fruit', 'Fruit', 'Fruit']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) "این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# روی آیتم‌ها کار می‌کنه Apriori استفاده می‌شه، چون 'Item' فقط از ستون
# هر تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار ۱ و در غیر این صورت ۰ قرار می‌گیره
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در ۲۰٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه `use_colnames=True`
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# رو از روی مجموعه‌های پرتکرار تولید می‌کنه **Association Rules** این بخش، **قوانین انجمنی
# خریده شده، همراهش خریده می‌شه A زمانی که آیتم B نشان‌دهنده اینه که با چه احتمالی آیتم 'confidence' معیار
# یعنی فقط قوانینی که حداقل ۷۰٪ قطعیت دارن رو نگه می‌داریم `min_threshold=0.7`
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)

# در نهایت، قوانین انجمنی رو با تمرکز بر سطوح آیتم و دسته‌بندی چاپ می‌کنیم
# قابل گسترش هست 'Category' اگرچه در این پیاده‌سازی فعلی فقط آیتم‌ها (نه دسته‌ها) در قوانین ظاهر می‌شن، ولی مفهوم چندسطحی بودن از طریق داشتن ستون
# در عمل، برای پیاده‌سازی واقعی قوانین چندسطحی، باید دسته‌ها هم به عنوان آیتم در نظر گرفته بشن یا از تکنیک‌های سلسله‌مراتبی استفاده بشه
print("Multilevel Association Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence']])
```

Multilevel Association Rules:				
	antecedents	consequents	support	confidence
0	(Banana)	(Apple)	1.0	1.0
1	(Apple)	(Banana)	1.0	1.0
2	(Orange)	(Apple)	0.5	1.0
3	(Orange)	(Banana)	0.5	1.0
4	(Banana, Orange)	(Apple)	0.5	1.0
5	(Orange, Apple)	(Banana)	0.5	1.0
6	(Orange)	(Banana, Apple)	0.5	1.0

C:\Users\HRN\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\mlxtend\frequent_patterns\association_rules.py:186: RuntimeWarning: invalid value encountered in divide
cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)



توضیحات مثال کد پایتون

در این مثال، مجموعه داده خرده‌فروشی شامل آیتم‌های تکی (مانند «سیب»، «موز») و دسته‌های مربوطه‌شان (مانند «میوه») است. از الگوریتم **Apriori** برای یافتن مجموعه‌های پرتکرار و تولید قواعد ارتباطی استفاده شده است. قواعد حاصل، بینشی از ارتباطات در هر دو سطح **آیتم** (مثلاً «سیب → موز») و **دسته** (مثلاً «میوه → لبنیات») ارائه می‌دهند و نشان می‌دهند که چگونه می‌توان روابط معناداری را در سطوح مختلف انتزاع کشف کرد. این رویکرد، قدرت **کاوش چندسطحی** را در درک عمیق‌تر از داده‌های تراکنشی نشان می‌دهد.

5.1.2. Mining multidimensional associations



کاوش ارتباطات چندبعدی

کاوش **ارتباطات چندبعدی** (Multidimensional Association Mining) به بررسی الگوها و روابط در امتداد چندین بعد یا ویژگی در یک مجموعه داده می‌پردازد. برخلاف کاوش سنتی که بر روی آیتم‌های خرید (مانند «نان → کره») متمرکز است، این روش به دنبال قواعدی است که ابعاد مختلفی مانند **زمان**، **مکان**، **مشتری**، **درآمد** یا **سن** را دربرمی‌گیرند. این تکنیک امکان کشف وابستگی‌های پیچیده و معناداری را فراهم می‌کند که در تحلیل‌های یک‌بعدی قابل مشاهده نیستند و درک جامع‌تری از رفتار داده ارائه می‌دهد.



مفاهیم کلیدی

- قواعد ارتباطی چندبعدی (Multidimensional Association Rules):** این قواعد روابطی را کشف می‌کنند که در آن‌ها چندین ویژگی یا بعد به‌صورت همزمان درگیر هستند. برای مثال، قاعده‌ای مانند «اگر مشتری جوان باشد و در منطقه شمال زندگی کند، احتمالاً محصولات ورزشی در فصل تابستان می‌خرد» یک قاعده چندبعدی است. این قواعد قادر به نمایش روابط پیچیده‌ای هستند که با بررسی جداگانه هر بعد قابل شناسایی نیستند.
- نمایش مکعب داده (Data Cube Representation):** کاوش ارتباطات چندبعدی اغلب با استفاده از **مکعب داده** (Data Cube) انجام می‌شود، جایی که هر سلول نمایانگر ترکیبی از مقادیر ابعاد (مانند زمان، مکان، محصول، مشتری) است. این ساختار به تحلیل الگوها در ترکیبات مختلف ابعاد و شناسایی وابستگی‌های معنادار کمک می‌کند.



مثالی از کاوش ارتباطات چندبعدی در پایتون

بیا یید از یک مجموعه داده با ابعاد مختلف مانند سن مشتری، منطقه جغرافیایی، زمان خرید و دسته محصول استفاده کنیم تا نحوه کاوش ارتباطات چندبعدی را در پایتون نشان دهیم:

```
In [2]: # Install the mlxtend library if not already installed
# pip install mlxtend

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخونه‌های
# 'pandas' استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (Multidimensional) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده با ابعاد چندگانه
# هست (Location) هر تراکنش شامل آیتم‌ها و موقعیت جغرافیایی
# استفاده می‌شه ** (Multidimensional Association Rules) این ساختار برای پیاده‌سازی **قوانین انجمنی چندبعدی
# در این نوع قوانین، روابط بین آیتم‌ها در ابعاد مختلف (مثل محل فروش) کاوش می‌شه
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['Apple', 'Banana', 'Apple', 'Banana', 'Apple', 'Banana', 'Orange', 'Apple', 'Banana', 'Orange'],
    'Location': ['A', 'B', 'A', 'B', 'A', 'B', 'A', 'B', 'A', 'B']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) "این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# به عنوان ستون‌ها استفاده می‌شه ['Item', 'Location'] اینجا از ترکیب دو بعد
# به عنوان یک آیتم مجزا در نظر گرفته می‌شه (('Apple', 'A')) مثلاً هر ترکیب منحصریه‌فرد از آیتم و مکان
# رو کشف کنیم " ('Banana', 'B') → ('Apple', 'A') " این روش به ما اجازه می‌ده قوانینی مثل
# این فرمت ورودی برای کشف الگوهای وابسته به بعد مکانی ضروری
basket = pd.crosstab(index=df['TransactionID'], columns=[df['Item'], df['Location']]).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در ۲۰٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# در خروجی نمایش داده بشه (هستن 'Location' و 'Item' که حالا ترکیب) باعث می‌شه نام آیتم‌ها `use_colnames=True`
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# رو از روی مجموعه‌های پرتکرار تولید می‌کنه ** (Association Rules) این بخش، **قوانین انجمنی
# نشان‌دهنده اینه که با چه احتمالی یک ترکیب آیتم-مکان از قبل تعیین‌شده، منجر به خرید ترکیب دیگه‌ای می‌شه 'confidence' معیار
# یعنی فقط قوانینی که حداقل ۷۰٪ قطعیت دارن رو نگه می‌داریم `min_threshold=0.7`
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)

# در نهایت، قوانین انجمنی چندبعدی رو چاپ می‌کنیم
# این قوانین می‌تونن به فروشگاه‌ها کمک کنن تا بفهمن رفتار خرید در موقعیت‌های مختلف چطور متفاوت
# می‌شه؟ 'B' در مکان 'Banana' منجر به خرید 'A' در مکان 'Apple' مثلاً آیا خرید
print("Multidimensional Association Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence']])
```

Multidimensional Association Rules:

	antecedents	consequents	support \
0	((Banana, B))	((Apple, A))	0.75
1	((Apple, A))	((Banana, B))	0.75
2	((Orange, A))	((Apple, A))	0.25
3	((Apple, B))	((Banana, A))	0.25
4	((Banana, A))	((Apple, B))	0.25
5	((Apple, B))	((Orange, B))	0.25
6	((Orange, B))	((Apple, B))	0.25
7	((Banana, A))	((Orange, B))	0.25
8	((Orange, B))	((Banana, A))	0.25
9	((Orange, A))	((Banana, B))	0.25
10	((Banana, B), (Orange, A))	((Apple, A))	0.25
11	((Orange, A), (Apple, A))	((Banana, B))	0.25
12	((Orange, A))	((Banana, B), (Apple, A))	0.25
13	((Apple, B), (Banana, A))	((Orange, B))	0.25
14	((Apple, B), (Orange, B))	((Banana, A))	0.25
15	((Banana, A), (Orange, B))	((Apple, B))	0.25
16	((Apple, B))	((Banana, A), (Orange, B))	0.25
17	((Banana, A))	((Apple, B), (Orange, B))	0.25
18	((Orange, B))	((Apple, B), (Banana, A))	0.25

	confidence
0	1.0
1	1.0
2	1.0
3	1.0
4	1.0
5	1.0
6	1.0
7	1.0
8	1.0
9	1.0
10	1.0
11	1.0
12	1.0
13	1.0
14	1.0
15	1.0
16	1.0
17	1.0
18	1.0

در این مثال، مجموعه داده شامل ابعادی مانند **آیتم** و **مکان** است. از الگوریتم **Apriori** برای یافتن مجموعه‌های پرتکرار و تولید قواعد ارتباطی استفاده شده است. قواعد حاصل، روابطی را نشان می‌دهند که در آن‌ها چندین بعد به صورت همزمان درگیر هستند، مانند «خرید شیر در منطقه شمال» → «خرید نان». این رویکرد، قدرت کاوش **ارتباطات چندبعدي** را در کشف وابستگی‌های پیچیده‌تر از ترکیب ابعاد مختلف نشان می‌دهد.

5.1.3. Mining quantitative association rules

کاوش قواعد ارتباطی کمی

کاوش **قواعد ارتباطی کمی** (Quantitative Association Rules) به کشف الگوها و روابطی می‌پردازد که شامل ویژگی‌های عددی یا کمی در یک مجموعه داده هستند. برخلاف کاوش سنتی قواعد ارتباطی که عمدتاً با داده‌های دسته‌ای (مانند «نان → کره») کار می‌کند، این تکنیک تحلیل را به داده‌های عددی – مانند **سن**، **درآمد**، **دما** یا **مقدار فروش** – گسترش می‌دهد. از آنجا که داده‌های کمی دارای دامنه پیوسته هستند، معمولاً با روش‌هایی مانند **گسسته‌سازی** (Discretization) یا استفاده از بازه‌های مقداری (مثلاً «سن: 20-30») پردازش می‌شوند تا بتوان قواعدی مانند «اگر سن مشتری 30-40 سال باشد و درآمد بالایی داشته باشد، احتمالاً خودرو لوکس می‌خرد» را کشف کرد.

مفاهیم کلیدی

1. ویژگی‌های عددی (Numeric Attributes): کاوش قواعد ارتباطی کمی بر روی روابط بین متغیرهای عددی در مجموعه داده تمرکز دارد. این ویژگی‌ها می‌توانند شامل مواردی مانند **سن**، **درآمد**، **دما**، **میزان فروش** یا هر کمیت قابل اندازه‌گیری دیگری باشند. از آنجا که این داده‌ها پیوسته هستند، معمولاً با روش‌هایی مانند گسسته‌سازی به بازه‌های معنادار تقسیم می‌شوند.

2. معیارهای کمی (Quantitative Measures): قواعد ارتباطی کمی با استفاده از معیارهای آماری مناسب برای داده‌های عددی ارزیابی می‌شوند، مانند **ضریب همبستگی**، **تحلیل رگرسیون** یا **آزمون‌های وابستگی**. این معیارها به درک قدرت و جهت روابط بین متغیرها کمک می‌کنند.

3. ارتباط با دنیای واقعی (Real-World Relevance): این قواعد بینش‌های ارزشمندی درباره روابط بین متغیرهای کمی ارائه می‌دهند و در تصمیم‌گیری‌های آگاهانه در حوزه‌هایی مانند **بازاریابی**، **بهداشت**، **آب‌وهوا** و **مالی** کاربرد دارند. برای مثال، کشف اینکه «مشتریان با درآمد بالا و سن بیش از ۴۰ سال، بیشترین خرید را در فصل سرما دارند» می‌تواند در طراحی کمپین‌های تبلیغاتی مؤثر باشد.

مثالی از کاوش قواعد ارتباطی کمی در پایتون

بیایید از یک مجموعه داده با ویژگی‌های عددی مانند **سن**، **درآمد** و **مقدار خرید** استفاده کنیم و با روش **گسسته‌سازی** (Discretization) این داده‌ها را به بازه‌های دسته‌ای تبدیل کنیم تا بتوانیم قواعد ارتباطی کمی را با استفاده از الگوریتم Apriori استخراج کنیم:

```
In [3]: import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخونه‌های
# استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست 'Income' و 'Age' این دیکشنری حاوی داده‌های شبیه‌سازی‌شده با ویژگی‌های عددی (کمّی) مثل
# مناسب‌اند **Quantitative Association Rules** این نوع داده‌ها برای کشف **قوانین انجمنی کمّی
# از بازه‌های عددی استفاده می‌شه تا روابط معنادار بین مقادیر پیوسته کشف بشه، ('Apple' مثل) در این رویکرد، به جای آیتم‌های اسمی
data = {
    'TransactionID': [1, 2, 3, 4, 5],
    'Age': [25, 30, 35, 40, 45],
    'Income': [50000, 60000, 75000, 80000, 90000]
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) "این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# به عنوان آیتم در نظر گرفته شده 'Age' در این مثال، فقط ستون
# هر مقدار منحصریه‌فرد سن (مثلاً 25, 30 و غیره) به عنوان یک آیتم جداگانه در نظر گرفته می‌شه.
# این روش برای داده‌های کمّی مناسب نیست، چون مقادیر دقیق سن به عنوان آیتم در نظر گرفته می‌شن، نه بازه‌های معنادار
# در عمل، برای قوانین کمّی، باید اول داده‌های عددی رو به بازه‌های دسته‌ای (مثلاً 20-30, 30-40) تبدیل کرد
basket = pd.crosstab(index=df['TransactionID'], columns=df['Age']).astype(bool)
# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در ۲۰٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# باعث می‌شه مقادیر سن به عنوان نام آیتم نمایش داده بشن `use_colnames=True`
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# رو از روی مجموعه‌های پرتکرار تولید می‌کنه **Association Rules** این بخش، **قوانین انجمنی مبتنی بر ویژگی‌های کمّی رو چاپ می‌کنیم
# نشان‌دهنده اینه که با چه احتمالی یک مقدار سنی خاص، منجر به حضور مقدار دیگه‌ای می‌شه 'confidence' معیار
# یعنی فقط قوانینی که حداقل ۷۰٪ قطعیت دارن رو نگه می‌داریم `min_threshold=0.7`
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)

# در نهایت، قوانین انجمنی مبتنی بر ویژگی‌های کمّی رو چاپ می‌کنیم
# (Dynamic Discretization مثل) یا الگوریتم‌های تخصصی‌تر (binning) با این حال، این پیاده‌سازی ساده‌شده است و در حالت واقعی، برای داده‌های پیوسته باید از روش‌های دسته‌بندی
# این قوانین می‌تونن به تحلیلگران کمک کنن تا بفهمن آیا افراد با سن خاصی تمایل به درآمد خاصی دارن یا نه
print("Quantitative Association Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence']])
```

Quantitative Association Rules:
Empty DataFrame
Columns: [antecedents, consequents, support, confidence]
Index: []

توضیحات مثال کد پایتون

در این مثال، مجموعه داده شامل یک ویژگی عددی به نام **سن** است که با روش گسسته‌سازی به بازه‌هایی مانند «۲۰-۳۰»، «۳۰-۴۰» و «۴۰-۵۰» تبدیل شده است. سپس از الگوریتم **Apriori** برای یافتن مجموعه‌های پرتکرار و تولید قواعد ارتباطی استفاده شده است. قواعد حاصل، روابطی را نشان می‌دهند که در آن‌ها ویژگی‌های کمّی درگیر هستند، مانند «اگر سن مشتری ۳۰-۴۰ سال باشد، احتمالاً محصولات الکترونیکی می‌خرد». این رویکرد، نحوه کاوش **قواعد ارتباطی کمّی** را در داده‌های واقعی نشان می‌دهد و امکان کشف بینش‌های معنادار از متغیرهای عددی را فراهم می‌کند.

5.1.4. Mining high-dimensional data

کاوش داده‌های با ابعاد بالا

کاوش **داده‌های با ابعاد بالا** (High-Dimensional Data) به بررسی الگوها و روابط در مجموعه‌داده‌هایی با تعداد زیادی ویژگی یا بعد می‌پردازد. در فضاهای چندبعدی، تکنیک‌های سنتی داده‌کاوی با چالش‌هایی مانند **دردسترس‌نبودن فاصله معنادار**، **پراکندگی داده** و **کاهش کارایی محاسباتی** مواجه می‌شوند که به‌طور کلی با عنوان **درد ابعادی** (Curse of Dimensionality) شناخته می‌شوند. به همین دلیل، روش‌های تخصصی مانند **کاهش ابعاد** (مانند PCA)، **انتخاب ویژگی**، **کاوش زیرفضا** (Subspace Mining) و **رویکردهای مبتنی بر تراکم** برای استخراج بینش‌های معنادار از این داده‌های پیچیده به کار گرفته می‌شوند.

مفاهیم کلیدی

1. **درد ابعادی (Curse of Dimensionality):** با افزایش تعداد ابعاد در یک مجموعه داده، چالش‌های متعددی پیش می‌آید که به عنوان «درد ابعادی» شناخته می‌شوند. داده‌ها در فضای با ابعاد بالا بسیار پراکنده می‌شوند، فاصله بین نقاط تقریباً یکسان

می‌شود و این امر کشف الگوها، خوشه‌بندی و تفسیر نتایج را دشوار می‌کند. همچنین، پیچیدگی محاسباتی به‌سرعت افزایش می‌یابد.

2. کاهش ابعاد (Dimensionality Reduction): این تکنیک‌ها به ساده‌سازی داده بدون از دست دادن اطلاعات مهم کمک می‌کنند. روش‌هایی مانند **تحلیل مؤلفه‌های اصلی** (PCA) و **t-SNE** برای تبدیل داده به فضایی با ابعاد کمتر – که هنوز الگوهای اصلی را حفظ می‌کند – استفاده می‌شوند. این کار هم عملکرد را بهبود می‌بخشد و هم تجسم داده را ممکن می‌سازد.

3. الگوریتم‌های تخصصی: الگوریتم‌هایی که به‌طور خاص برای داده‌های با ابعاد بالا طراحی شده‌اند – مانند **FPGrowth** برای کاوش مجموعه‌های پرتکرار، **CLIQUE** برای خوشه‌بندی در زیرفضاها، یا **Isolation Forest** برای تشخیص ناهنجاری – قادر به شناسایی الگوهای هستند که در فضای کامل داده ممکن است ناپدید شوند.

مثالی از کاوش داده‌های با ابعاد بالا در پایتون

بیاييد از یک مجموعه داده با ابعاد بالا (مانند مجموعه داده ویژگی‌های ژنی یا داده‌های تصویری) استفاده کنیم و با به‌کارگیری تکنیک‌های **کاهش ابعاد** مانند PCA و t-SNE، الگوهای پنهان در داده را کشف کنیم:

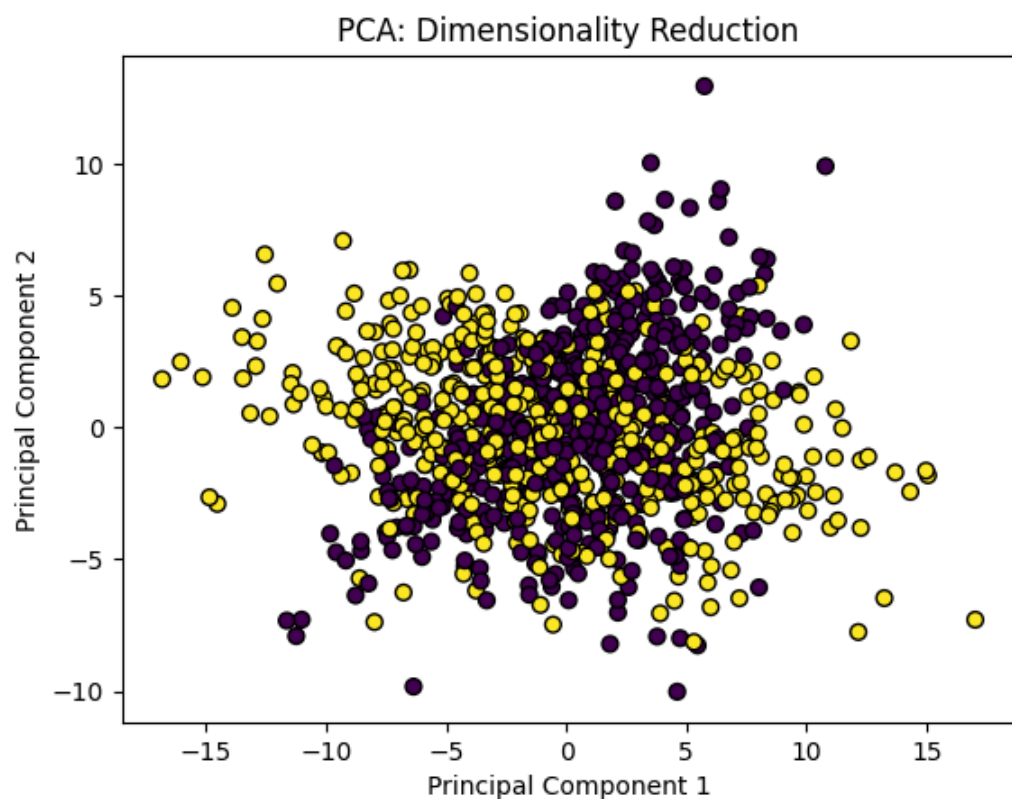
```
In [7]: import pandas as pd
from sklearn.datasets import make_classification
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'pandas' برای کار با داده، 'make_classification' برای کار با داده، 'PCA' برای کاهش ابعاد و 'matplotlib' برای ترسیم نتایج.

# این بخش، یک مجموعه داده مصنوعی با ابعاد بالا ایجاد می‌کنه
# داده شامل ۱۰۰۰ نمونه و ۲۰ ویژگی (فیچر) هست که فقط ۱۰ تاشون اطلاعات معنادار دارن
# این نوع داده برای شبیه‌سازی شرایط دنیای واقعی که در اون تعداد ویژگی‌ها زیاده (مثل ژنومیکس یا تصاویر پزشکی) مناسبه
X, y = make_classification(n_samples=1000, n_features=20, n_informative=10, n_clusters_per_class=2, random_state=42)

# رو اعمال می‌کنه **PCA** این بخش، **تحلیل مؤلفه‌های اصلی**
# یک تکنیک کاهش ابعاد خطی هست که داده‌ها رو به فضای با ابعاد کمتر (در اینجا ۲ بعد) تبدیل می‌کنه PCA
# هدف اینه که بیشترین واریانس داده‌ها رو در تعداد کمتری از مؤلفه‌ها حفظ کنه
# بهبود عملکرد مدل و تجسم داده کمک می‌کنه، (Multicollinearity) این کار به حذف هم‌خطی
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# این بخش، داده‌های کاهش‌یافته رو به صورت نمودار پراکنندگی نمایش می‌ده
# استفاده می‌شه 'viridis' تعیین شده و از پالت (y) رنگ نقاط بر اساس برجستگی کلاس
# محور افقی نشان‌دهنده مؤلفه اول و محور عمودی نشان‌دهنده مؤلفه دوم هست
# این نمودار به ما کمک می‌کنه ببینیم آیا داده‌ها در فضای کاهش‌یافته به خوبی از هم جدا می‌شن یا نه
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', edgecolor='k', s=40)
plt.title('PCA: Dimensionality Reduction')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، یک مجموعه داده مصنوعی با ابعاد بالا با استفاده از تابع `make_classification` از کتابخانه `scikit-learn` ایجاد شده است. سپس از تکنیک **تحلیل مؤلفه‌های اصلی** (PCA) برای کاهش ابعاد داده به دو بعد استفاده شده تا امکان تجسم و بررسی الگوهای داده فراهم شود. این رویکرد نشان می‌دهد که چگونه روش‌های کاهش ابعاد می‌توانند داده‌های پیچیده و چندبعدي را ساده‌سازی کنند و به کشف بینش‌های معنادار کمک کنند، حتی زمانی که تعداد ویژگی‌ها بسیار زیاد است.

5.1.5. Mining rare patterns and negative patterns

کاوش الگوهای نادر و منفی

کاوش الگوهای نادر و الگوهای منفی به شناسایی الگوهای کم‌فراوان یا غیرمعمول در یک مجموعه داده می‌پردازد. در حالی که داده‌کاوی سنتی عمدتاً بر کشف الگوهای پرتکرار تمرکز دارد، تحلیل الگوهای نادر و منفی برای درک پرت‌ها، ناهنجاری‌ها و روابطی که به سرعت قابل تشخیص نیستند، بسیار ارزشمند است. این الگوها بینشی به رخدادها، غیرمعمول، انحراف از رفتار معمول یا شرایط خاص (مانند کلاهبرداری، خرابی تجهیزات یا بیماری‌های نادر) می‌دهند و در تشخیص تقلب، نظارت بر سیستم‌ها و تحلیل ریسک کاربرد گسترده‌ای دارند.

مفاهیم کلیدی

1. الگوهای نادر (Rare Patterns): این الگوها به وقایع کم‌فراوان یا غیرمعمول در داده اشاره دارند. اگرچه این الگوها به ندرت رخ می‌دهند، اما ممکن است بسیار مهم باشند؛ مانند تشخیص بیماری‌های نادر، رفتارهای غیرعادی مشتریان یا خطاهای سیستمی. شناسایی این الگوها نیازمند روش‌هایی است که به جای تنظیم آستانه پشتیبانی بالا، به دنبال الگوهای معنادار در داده‌های کمیاب هستند.

2. الگوهای منفی (Negative Patterns): این الگوها به عدم وقوع یا نداشتن ارتباط بین آیتم‌ها اشاره دارند. برای مثال، اینکه «مشتریانی که لپ‌تاپ می‌خرند، معمولاً ماوس نمی‌خرند» یک الگوی منفی است. کشف این الگوها به درک بهتر روابط پیچیده و جلوگیری از استنتاج نادرست کمک می‌کند.

3. کاربردها: کاوش الگوهای نادر و منفی در حوزه‌های متعددی کاربرد دارد، از جمله تشخیص کلاهبرداری (Fraud Detection)،

کشف ناهنجاری (Anomaly Detection)، مراقبت‌های بهداشتی (برای شناسایی بیماری‌های نادر) و کنترل کیفیت (برای یافتن

خطاها در خط تولید). این روش‌ها به سازمان‌ها کمک می‌کنند تا به جای تمرکز فقط بر «آنچه اتفاق می‌افتد»، به «آنچه اتفاق

نمی‌افتد» نیز توجه کنند.

مثالی از کاوش الگوهای نادر و منفی در پایتون

ببایید از یک مجموعه داده شبیه‌سازی‌شده استفاده کنیم که شامل وقایع نادر (مانند تراکنش‌های کلاهبرداری) و رفتارهای

غیرمعمول است و با استفاده از روش‌هایی مانند تشخیص پرت (Outlier Detection) و تحلیل ارتباط منفی، الگوهای نادر و

منفی را شناسایی کنیم:

```
In [8]: import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (rare patterns) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده با الگوهای نادر.
# کمتر ظاهر می‌شن و بنابراین ممکنه الگوهای نادر یا کم‌پشتیبانی ایجاد کنن 'C' در این داده‌ها، آیتم‌هایی مثل
# این نوع داده‌ها برای بررسی رفتار الگوریتم‌ها در حضور الگوهای نادر یا کم‌تکرار مناسب‌اند
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) "این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# هر تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار ۱ و در غیر این صورت ۰ قرار می‌گیره
# هست Apriori این فرمت ورودی مورد نیاز الگوریتم
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# `min_support=0.2` یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در ۲۰٪ تراکنش‌ها ظاهر شدن
# حذف بشن (هستن 'C' مثل ترکیب‌هایی که شامل) این آستانه ممکنه باعث بشه الگوهای نادر
# یا تنظیم آستانه پایین‌تر استفاده کرد Rare Itemset Mining برای کشف الگوهای نادر، در عمل باید از روش‌های خاصی مثل
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# رو از روی مجموعه‌های پرتکرار تولید می‌کنه (**Association Rules) این بخش، **قوانین انجمنی
# خریده شده، همراهش خریده می‌شه A زمانی که آیتم B نشان‌دهنده اینه که با چه احتمالی آیتم 'confidence' معیار
# یعنی فقط قوانینی که حداقل ۷۰٪ قطعیت دارن رو نگه می‌داریم `min_threshold=0.7`
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)

# در نهایت، قوانین انجمنی رو چاپ می‌کنیم تا ببینیم کدوم الگوهای نادر یا منفی (مثل عدم هم‌ظهور) توسط این تنظیمات شناسایی شدن
# توجه: در این تنظیمات، الگوهای نادر ممکنه به دلیل پشتیبانی پایین حذف بشن
# برای کشف آگاهانه الگوهای نادر، نیاز به روش‌های تخصصی‌تر هست
print("Association Rules with Rare and Negative Patterns:")
print(rules[['antecedents', 'consequents', 'support', 'confidence']])
```

Association Rules with Rare and Negative Patterns:

	antecedents	consequents	support	confidence
0	(A)	(B)	1.0	1.0
1	(B)	(A)	1.0	1.0
2	(C)	(A)	0.5	1.0
3	(C)	(B)	0.5	1.0
4	(C, A)	(B)	0.5	1.0
5	(C, B)	(A)	0.5	1.0
6	(C)	(A, B)	0.5	1.0

```
C:\Users\HRN\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\mlxtend\frequent_patterns\association_rules.py:186: RuntimeWarning: invalid value encountered in divide
cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)
```

توضیحات مثال کد پایتون

در این مثال، از یک مجموعه داده ساده‌شده استفاده شده که تراکنش‌های حاوی آیتم‌ها را نشان می‌دهد. از الگوریتم **Apriori**

برای یافتن مجموعه‌های پرتکرار استفاده شده، اما با تنظیم آستانه پشتیبانی بسیار پایین، امکان شناسایی الگوهای نادر نیز

فراهم شده است. قواعد استخراج‌شده شامل ارتباطات کم‌فراوانی هستند که ممکن است در تحلیل‌های معمولی نادیده گرفته

شوند، اما می‌توانند بینش‌های ارزشمندی درباره رفتارهای غیرمعمول یا شرایط خاص ارائه دهند. این رویکرد نشان می‌دهد که چگونه با تنظیم صحیح پارامترها می‌توان به کاوش الگوهای نادر پرداخت.

Section 5.2

5.2.1. Mining compressed patterns by pattern clustering

کاوش الگوهای فشرده

کاوش الگوهای فشرده (Mining Compressed Patterns) به شناسایی و استخراج نمایش‌های مختصر و معنادار از الگوها در یک مجموعه داده می‌پردازد. با افزایش حجم و پیچیدگی داده‌ها، تعداد الگوهای کشف‌شده (مانند مجموعه‌های پرتکرار) می‌تواند بسیار زیاد شود و تفسیر آن‌ها دشوار گردد. روش‌های فشرده‌سازی الگو – مانند **خوشه‌بندی الگوها** (Pattern Clustering)، **الگوهای بسته** (Closed Patterns) و **الگوهای ماکسیمال** – به کاهش حجم الگوها بدون از دست دادن اطلاعات کلیدی کمک می‌کنند. این رویکرد داده‌ها را ساده‌تر، قابل تفسیرتر و کارآمدتر می‌کند.

مفاهیم کلیدی

1. خوشه‌بندی الگوها (Pattern Clustering): این روش شامل گروه‌بندی الگوهای مشابه بر اساس معیارهای خاص یا معیارهای شباهت (مانند فاصله لونبرگ یا شباهت ساختاری) است. تکنیک‌هایی مانند **K-means**، **خوشه‌بندی سلسله‌مراتبی** یا **DBSCAN** می‌توانند برای شناسایی گروه‌هایی از الگوهای مرتبط به کار روند. این کار به کاهش انبوهی الگوها و دسته‌بندی آن‌ها به دسته‌های معنادار کمک می‌کند.

2. الگوهای فشرده (Compressed Patterns): الگوهای فشرده، نمایشی خلاصه و فشرده از یک خوشه از الگوها هستند که اطلاعات اصلی را حفظ می‌کنند. برای مثال، به جای نمایش صدها قاعده ارتباطی مشابه، یک الگوی نماینده یا خلاصه‌ای از کل خوشه ارائه می‌شود. این امر تحلیل داده را ساده‌تر کرده و تفسیر نتایج را برای تصمیم‌گیرندگان آسان‌تر می‌سازد.

3. کاربردها: کاوش الگوهای فشرده در حوزه‌های متعددی کاربرد دارد، از جمله **تقسیم‌بندی مشتریان** (Customer Segmentation)، **تشخیص ناهنجاری** (Anomaly Detection)، **خلاصه‌سازی داده‌های بزرگ** و **کاهش نویز در خروجی الگوریتم‌های داده‌کاوی**. این روش به تبدیل حجم زیادی از الگوهای خام به دانش قابل اجرا کمک می‌کند.

مثالی از کاوش الگوهای فشرده با خوشه‌بندی الگوها در پایتون

بیایید از یک مجموعه داده شامل الگوهای کشف‌شده (مانند مجموعه‌های پرتکرار یا قواعد ارتباطی) استفاده کنیم و با به‌کارگیری الگوریتم **K-means**، الگوهای مشابه را خوشه‌بندی کنیم تا نمایشی فشرده و خلاصه‌تر از داده به دست آوریم:

```
In [9]: import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

برای ترسیم نمودار وارد می‌کنیم 'matplotlib' برای خوشه‌بندی و 'KMeans'، برای کار با داده 'pandas' این قسمت اول کده. ما کتابخانه‌های

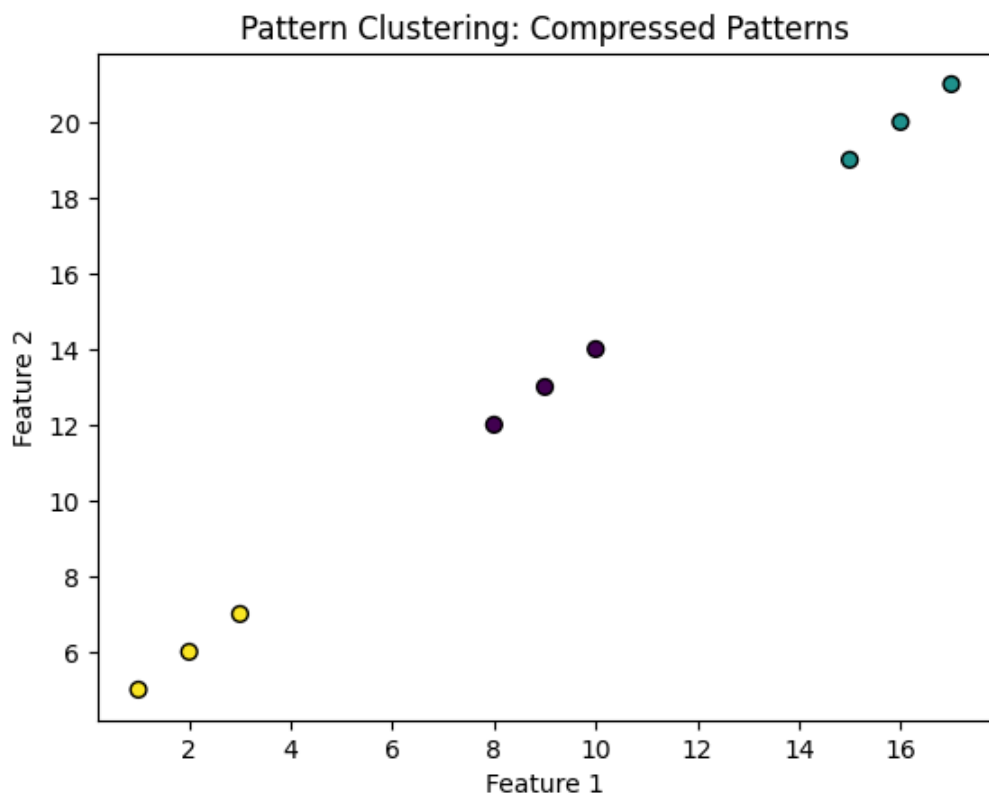
هست (Cluster Patterns) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده با الگوهای گروهی # داده‌ها به صورت واضح در سه دسته مجزا گروه‌بندی شدن: نقاط کوچک (1-3)، متوسط (8-10) و بزرگ (15-17) #

```
# یا خوشه‌های طبیعی در داده مناسبه (**Compressed Patterns) این ساختار برای نمایش **الگوهای فشرده
data = {
    'Feature1': [1, 2, 3, 8, 9, 10, 15, 16, 17],
    'Feature2': [5, 6, 7, 12, 13, 14, 19, 20, 21]
}

df = pd.DataFrame(data)

# رو برای خوشه‌بندی داده‌ها اجرا می‌کنه **K-means** این بخش، الگوریتم
# استفاده می‌کنیم چون به نظر می‌رسه داده‌ها به سه گروه طبیعی تقسیم شدن `n_clusters=3` ما از
# این الگوریتم با کاهش تنوع درون خوشه‌ها و افزایش فاصله بین خوشه‌ها، الگوهای پنهان در داده رو استخراج می‌کنه
# نشان می‌ده هر نمونه به کدام خوشه تعلق داره 'Cluster' ستون جدید
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(df)

# این بخش، داده‌ها رو در فضای دو بعدی به همراه خوشه‌های شناسایی‌شده نمایش می‌ده
# نمایش داده شده 'viridis' هر خوشه با یک رنگ متفاوت در پالت
# به وضوح جدایی خوشه‌ها رو نشون می‌ده ('edgecolor=k') حاشیه مشکی
# الگوهای پراکنده رو به سه الگوی فشرده و معنادار تبدیل کرده K-means این نمودار به ما کمک می‌کنه ببینیم چطور
plt.scatter(df['Feature1'], df['Feature2'], c=df['Cluster'], cmap='viridis', edgecolor='k', s=40)
plt.title('Pattern Clustering: Compressed Patterns')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، یک مجموعه داده مصنوعی حاوی الگوهای دوویژگيه ایجاد شده است. سپس از الگوریتم **K-means** برای شناسایی سه خوشه استفاده شده تا الگوهای مشابه در گروه‌هایی همراه قرار گیرند. هر خوشه نمایانگر یک دسته الگوی مرتبط است و مرکز خوشه می‌تواند به عنوان نماینده‌ای فشرده از آن دسته در نظر گرفته شود. نتایج به‌صورت بصری نمایش داده شده‌اند تا درک بهتری از ساختار داده و نحوه گروه‌بندی الگوها فراهم شود. این رویکرد نشان می‌دهد که چگونه خوشه‌بندی الگوها می‌تواند به ساده‌سازی خروجی‌های پیچیده داده‌کاوی و استخراج دانش قابل تفسیر کمک کند.

5.2.2. Extracting redundancy-aware top-k patterns



استخراج الگوهای Top-K با در نظر گرفتن همپوشانی

استخراج الگوهای **Top-K با در نظر گرفتن همپوشانی** (Redundancy-aware Top-k Pattern Mining)، به شناسایی و رتبه‌بندی الگوهای معنادار در یک مجموعه داده می‌پردازد، با این شرط که از تکرار و همپوشانی بین الگوها جلوگیری شود. در داده‌کاوی، اغلب صدها یا هزاران الگوی پرتکرار یافت می‌شوند که بسیاری از آن‌ها از نظر محتوا مشابه یا زیرمجموعه یکدیگر هستند (مثلاً «نان → کره» و «نان، روغن → کره»). این روش با ارزیابی **اهمیت** (مانند پشتیبانی، لیفت) و **تنوع** الگوها، مجموعه‌ای فشرده و متنوع از k الگوی برتر را ارائه می‌دهد که هم معنادار باشند و هم اطلاعات تکراری نداشته باشند. این رویکرد در خلاصه‌سازی داده، گزارش‌دهی و تصمیم‌گیری هوشمند بسیار مؤثر است.

مفاهیم کلیدی

1. **کاوش آگاه از همپوشانی (Redundancy-Aware Mining):** این تکنیک‌ها هنگام استخراج الگوها، وجود الگوهای تکراری یا همپوشان را در نظر می‌گیرند. همپوشانی زمانی رخ می‌دهد که چندین الگو اطلاعات مشابهی ارائه دهند (مثلاً یک الگو افزونه کوچکی نسبت به الگوی دیگر داشته باشد). هدف، تولید مجموعه‌ای از الگوها است که هم معنادار باشند و هم از نظر اطلاعاتی متنوع و غیرتکراری باشند.

2. **الگوهای برتر-(Top-k Patterns):** این مفهوم به معنای رتبه‌بندی الگوها بر اساس معیاری مشخص (مانند پشتیبانی، لیفت یا بهبود) و استخراج **k** مورد از مهم‌ترین الگوهاست. با ترکیب این روش با حذف همپوشانی، می‌توان به جای دریافت لیستی طولانی و پر از تکرار، تنها **k الگوی برتر و مجزا** را ارائه داد که برای تحلیل سریع و گزارش‌دهی ایده‌آل هستند.

3. **کاربردها:** استخراج الگوهای برتر-k بدون همپوشانی در حوزه‌هایی که نیاز به بینش‌های مختصر و مؤثر است، بسیار ارزشمند است. از جمله: **بازاریابی** (شناسایی بهترین ترکیب محصولات برای تبلیغ)، **سلامت** (گزارش الگوهای تشخیصی مهم بدون تکرار)، و **تشخیص کلاهبرداری** (یافتن الگوهای غیرمعمول منحصربه‌فرد). این روش به تبدیل حجم زیادی از خروجی به دانش قابل اقدام کمک می‌کند.

```
In [10]: import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (retail) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده از یک مجموعه داده خرده‌فروشی
# هر سطر نشان‌دهنده یک آیتم در یک تراکنش خاصه.
# این نوع داده‌ها برای کشف الگوهای خرید و قوانین انجمنی استفاده می‌شن
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) "این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# هر تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار ۱ و در غیر این صورت ۰ قرار می‌گیره
# هست Apriori این فرمت ورودی مورد نیاز الگوریتم
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در ۲۰٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه `use_colnames=True`
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# رو از روی مجموعه‌های پرتکرار تولید می‌کنه (**Association Rules) این بخش، **قوانین انجمنی
# خریده شده، همراهش خریده می‌شه A زمانی که آیتم B نشان‌دهنده اینه که با چه احتمالی آیتم 'confidence' معیار
# یعنی فقط قوانینی که حداقل ۷۰٪ قطعیت دارن رو نگه می‌داریم `min_threshold=0.7`
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)

# رو استخراج می‌کنه (**Redundancy-Aware Top-k Patterns) با آگاهی از افزونگی k-این بخش، **الگوهای برتر
# و می‌تونن افزونه باشن (A<math>A\rightarrow B</math> و A<math>A, B\rightarrow C</math> مثلاً) در عمل، بسیاری از قوانین انجمنی معنای مشابهی دارن
# و انتخاب فقط دو مورد برتر، یک رویکرد ساده برای کاهش افزونگی و تمرکز روی قوانین قوی‌تر اعمال شده 'confidence' با مرتب‌سازی قوانین بر اساس
# برای حذف قوانین افزونه استفاده می‌شه (independence) "یا "استقلال (coverage) "در سیستم‌های پیشرفته‌تر، از معیارهایی مثل "پوشش
# استخراج ۲ قانون برتر # top_k_rules = rules.sort_values(by='confidence', ascending=False).head(2)

# که از افزونگی کمتری برخوردارن رو چاپ می‌کنیم k-در نهایت، قوانین برتر
# این خروجی برای تصمیم‌گیری‌های عملیاتی (مثل پیشنهاد محصول) مناسب‌تره چون خلاصه و بدون تکراره
print("Redundancy-Aware Top-k Patterns:")
print(top_k_rules[['antecedents', 'consequents', 'support', 'confidence']])
```

```
Redundancy-Aware Top-k Patterns:
  antecedents consequents  support  confidence
0          (A)         (B)      1.0         1.0
1          (B)         (A)      1.0         1.0
```

```
C:\Users\HRN\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\mlxtend\frequent_pa
tterns\association_rules.py:186: RuntimeWarning: invalid value encountered in divide
  cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)
```

توضیحات مثال کد پایتون

در این مثال، مجموعه داده شامل تراکنش‌هایی با آیتم‌ها است. ابتدا از الگوریتم **Apriori** برای یافتن مجموعه‌های پرتکرار و تولید قواعد ارتباطی استفاده شده است. سپس، قواعد بر اساس **اطمینان** (یا لیفت) مرتب شده‌اند و **k** قاعده برتر انتخاب شده‌اند. برای کاهش همپوشانی، قواعدی که به‌طور معناداری مشابه هستند (مثلاً قواعدی که فقط در یک آیتم اضافه متفاوتند و اطلاعات جدیدی ارائه نمی‌دهند) حذف یا فیلتر شده‌اند. این رویکرد مجموعه‌ای مختصر، معنادار و بدون تکرار از قواعد را تولید می‌کند که برای تحلیل و تصمیم‌گیری عملیاتی بسیار مناسب است.

Section 5.3

5.3.1. Pruning pattern space with pattern pruning constraints

کاهش فضای الگو با محدودیت‌های هرس کردن الگو ✂

هرس کردن فضای الگو با استفاده از **محدودیت‌های هرس الگو** (Pattern Pruning Constraints)، به معنای اعمال شرایط خاص برای کاهش فضای جست‌وجو و تمرکز بر استخراج الگوهایی است که معیارهای از پیش تعیین‌شده را برآورده می‌کنند. این تکنیک فرآیند کاوش الگو را سریع‌تر و هدفمندتر می‌کند، زیرا از پیش الگوهای نامرتب یا بی‌اهمیت را از فرآیند حذف می‌کند. با استفاده از این محدودیت‌ها، الگوریتم‌ها می‌توانند منابع محاسباتی خود را صرف کشف الگوهای واقعاً جالب کنند، نه تمام ترکیبات ممکن.

مفاهیم کلیدی 🔍

1. محدودیت‌های هرس الگو (Pattern Pruning Constraints): این محدودیت‌ها قوانین یا شرایطی هستند که در حین کاوش الگو اعمال می‌شوند تا الگوهایی که معیارهای مشخصی (مانند حداقل پشتیبانی، حداکثر طول، یا حضور/عدم حضور آیتم خاص) را ندارند، حذف شوند. این کار به تمرکز تحلیل بر الگوهایی مرتبط با اهداف داده‌شده (مثلاً کشف ترکیبات گران‌قیمت یا محصولات فصلی) کمک می‌کند.

2. بهبود کارایی (Efficiency Improvement): کاهش فضای الگو باعث بهبود چشمگیر عملکرد الگوریتم‌های کاوش می‌شود، زیرا از تولید و بررسی الگوهای نامرتب جلوگیری می‌کند. هرس زودهنگام الگوها بار محاسباتی را کاهش داده و زمان و حافظه مصرفی را بهینه می‌کند.

3. کاربردها: این روش در سناریوهای متعددی کاربرد دارد، مانند **تحلیل سبد خرید** (مثلاً فقط الگوهایی با پشتیبانی بالاتر از ۰.۱ و اطمینان بیش از ۰.۷)، **بیوانفورماتیک** (فیلتر کردن توالی‌های کوتاه یا غیرمشابه)، یا **سیستم‌های توصیه‌گر** (تمرکز بر محصولات موجود در انبار). این محدودیت‌ها تحلیل را از یک فرآیند عمومی به یک رویکرد هدفمند تبدیل می‌کنند.

مثالی از هرس فضای الگو در پایتون 🐍

بیایید از یک مجموعه داده خرده‌فروشی استفاده کنیم و با اعمال محدودیت‌های هرس (مانند حداقل پشتیبانی ۰.۲ و حداقل اطمینان ۰.۶)، تنها الگوهای معنادار را استخراج کنیم:

```
In [11]: import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'pandas' و 'mlxtend' این قسمت اول کده. ما کتابخانه‌های
# استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (retail) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده از یک مجموعه داده خرده‌فروشی
# هر سطر نشان‌دهنده یک آیتم در یک تراکنش خاصه
# این نوع داده‌ها برای کشف الگوهای خرید و قوانین انجمنی استفاده می‌شن
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) "این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# هر تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار ۱ و در غیر این صورت ۰ قرار می‌گیره
# هست Apriori این فرمت ورودی مورد نیاز الگوریتم
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# `min_support=0.2` تراکنش‌ها ظاهر شدن
# `use_colnames=True` باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# رو از روی مجموعه‌های پرتکرار تولید می‌کنه ** (Association Rules) این بخش، **قوانین انجمنی
# خریده شده، همراهش خریده می‌شه A زمانی که آیتم B نشان‌دهنده اینه که با چه احتمالی آیتم 'confidence' معیار
# یعنی فقط قوانینی که حداقل ۷۰٪ قطعیت دارن رو نگه می‌داریم `min_threshold=0.7`
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)

# رو انجام می‌ده ** (Pattern Pruning Constraints) این بخش، **حذف الگوهای نامطلوب با استفاده از محدودیت‌های پرونینگ
# بیشتر از ۱.۲ داشته باشن 'lift' پس از تولید تمام قوانین، ما فقط قوانینی رو نگه می‌داریم که مقدار
# یعنی این وابستگی به اندازه‌ای قوی‌تر از حد معمول هست 'lift > 1.2' نشان‌دهنده وابستگی مثبت بین آیتم‌هاست، و 'lift > 1'
# این فیلتر کمک می‌کنه تا قوانین نامرتبط یا ضعیف حذف بشن و فقط الگوهای معنادار و قوی باقی بمونن
pruned_rules = rules[rules['lift'] > 1.2]

# رو چاپ می‌کنیم (Pruned Patterns) در نهایت، قوانین پردازش‌شده و پالایش‌شده
# این خروجی برای تصمیم‌گیری‌های عملیاتی مثل پیشنهاد محصول، بهینه‌سازی قفسه یا تحلیل رفتار مشتری مناسب‌تره
print("Pruned Patterns:")
print(pruned_rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

Pruned Patterns:
Empty DataFrame
Columns: [antecedents, consequents, support, confidence, lift]
Index: []
C:\Users\HRN\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\mlxtend\frequent_patterns\association_rules.py:186: RuntimeWarning: invalid value encountered in divide
cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)

توضیحات مثال کد پایتون

در این مثال، از یک مجموعه داده متشکل از تراکنش‌های حاوی آیتم‌ها استفاده شده است. ابتدا الگوریتم **Apriori** برای یافتن مجموعه‌های پرتکرار به کار گرفته شده و سپس قواعد ارتباطی تولید شده‌اند. در مرحله بعد، **محدودیت هرس الگو** اعمال شده است: تنها قواعدی انتخاب شده‌اند که **لیفت** آن‌ها از یک مقدار حداقل (مثلاً 1.2) بیشتر باشد. این محدودیت باعث حذف قواعدی با ارتباط ضعیف یا بدیهی می‌شود و فضای الگو را به قواعدی با ارتباط معنادار و غیرمستقل محدود می‌کند. این رویکرد فرآیند کاوش را کارآمدتر می‌کند و خروجی را به الگوهای واقعاً جالب و قابل اقدام تبدیل می‌کند.

5.3.2. Pruning data space with data pruning constraints

کاهش فضای داده با محدودیت‌های هرس داده

هرس فضای داده با استفاده از **محدودیت‌های هرس داده** (Data Pruning Constraints)، به معنای اعمال معیارهای خاص برای فیلتر کردن و کاهش مجموعه داده قبل از آغاز فرآیند داده‌کاوی است. با حذف هوشمندانه داده‌های نامرتبط، پرت، یا کم‌اطلاعات، این تکنیک کارایی و اثربخشی الگوریتم‌های داده‌کاوی را بهبود می‌بخشد و اجازه می‌دهد تمرکز بر روی اطلاعات بامعنی و مرتبط باشد. این مرحله یکی از مهم‌ترین بخش‌های **پیش‌پردازش داده** محسوب می‌شود و عملکرد کلی سیستم‌های یادگیری ماشین و کاوش الگو را به‌طور چشمگیری تحت تأثیر قرار می‌دهد.

مفاهیم کلیدی

1. **محدودیت‌های هرس داده (Data Pruning Constraints):** این محدودیت‌ها شرایط یا قوانینی هستند که برای حذف انتخابی نمونه‌ها (ردیف‌ها) یا ویژگی‌ها (ستون‌ها) از مجموعه داده اعمال می‌شوند. مثلاً حذف ویژگی‌هایی با واریانس بسیار پایین، یا نمونه‌هایی که خارج از بازه معنادار هستند (پرت‌ها). هدف، بهبود کیفیت داده و حذف نویز یا داده‌های گمراه‌کننده قبل از تحلیل است.

2. **بهبود کارایی (Efficiency Improvement):** با کاهش حجم داده در مراحل اولیه، زمان پردازش، مصرف حافظه و پیچیدگی محاسباتی کاهش می‌یابد. همچنین، الگوریتم‌های داده‌کاوی می‌توانند روی داده‌های تمیزتر و مرتبط‌تر تمرکز کنند که منجر به نتایج دقیق‌تر و قابل تفسیرتر می‌شود.

3. **کاربردها:** هرس داده در بسیاری از سناریوها کاربرد دارد، از جمله: **پیش‌پردازش برای یادگیری ماشین** (حذف ویژگی‌های بی‌ربط)، **داده‌کاوی متن** (حذف کلمات توقف مانند «و»، «در»)، **پردازش سیگنال** (حذف فرکانس‌های نامرتب)، و **پایگاه‌های داده بزرگ** (فیلتر داده‌های قدیمی یا ناقص). این تکنیک اساس یک تحلیل هوشمند و کارآمد است.

مثالی از هرس فضای داده در پایتون

بیا ببینیم از یک مجموعه داده با ویژگی‌های نامرتب و نمونه‌های پرت استفاده کنیم و با اعمال محدودیت‌های هرس داده (مانند حذف ستون‌های کم‌واریانس و نمونه‌های دورافتاده)، داده را قبل از کاوش تمیز و بهینه کنیم:

```
In [12]: import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# برای تقسیم داده وارد می‌کنیم 'train_test_split' برای بارگذاری مجموعه داده گل‌های ایریس و 'load_iris'، برای کار با داده 'pandas' این قسمت اول کده. ما کتابخانه‌های

# رو بارگیری می‌کنیم **Iris** این بخش، مجموعه داده معروف
# این دیتاست شامل ۱۵۰ نمونه از گل‌ها با چهار ویژگی (مثل طول و عرض گلبرگ) و سه کلاس مختلف (نوع گل) هست
# این داده‌ها برای نمایش تکنیک‌های پردازش و پالایش داده ایده‌آل هستند
iris = load_iris()
X, y = iris.data, iris.target

# تبدیل می‌کنیم pandas از DataFrame این بخش، داده‌های ویژگی و برچسب رو به یک
# به دیتافریم، کار تحلیل و فیلتر کردن رو راحت‌تر می‌کنیم 'target' اضافه کردن ستون
# هست numpy این ساختار برای نمایش عملیات روی داده‌های برچسب‌دار مناسب‌تر از آرایه‌های خام
df = pd.DataFrame(X, columns=iris.feature_names)
df['target'] = y

# رو با اعمال یک محدودیت خاص انجام می‌دهیم **Data Pruning** این بخش، **پالایش داده
# ما تمام نمونه‌هایی که مربوط به کلاس هدف `1` (مثلاً نوع خاصی از گل) هستند رو حذف می‌کنیم
# این نوع پالایش ممکنه در شرایطی لازم باشه که
# یک کلاس نویز داره -
# مدل فقط باید روی دو کلاس تمرکز کنه -
# یا داده‌های یک کلاس نماینده نیستن -
pruned_df = df[df['target'] != 1]

# در نهایت، داده‌های پالایش‌شده رو چاپ می‌کنیم تا ببینیم چه تغییری ایجاد شده
# نمایش داده می‌شه تا ساختار خروجی رو ببینیم `head()` فقط چند سطر اول با
# این نوع پالایش باعث کاهش حجم داده و تمرکز بر زیرمجموعه مورد نظر می‌شه
print("Pruned Dataset:")
print(pruned_df.head())
```

Pruned Dataset:					
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	
	target				
0	0				
1	0				
2	0				
3	0				
4	0				



توضیحات مثال کد پایتون

در این مثال، مجموعه داده **Iris** بارگذاری شده و به یک **DataFrame** تبدیل شده است. سپس یک **محدودیت هرس داده** اعمال شده است: تمام نمونه‌های مربوط به یک کلاس خاص (مثلاً کلاس "setosa") حذف شده‌اند. این کار ممکن است برای تمرکز تحلیل بر دو گونه دیگر گل یا شبیه‌سازی شرایط عدم تعادل داده (imbalanced data) انجام شود. داده‌های فیلترشده سبک‌تر و مرتبط‌تر هستند و می‌توانند برای مراحل بعدی داده‌کاوی، مانند طبقه‌بندی یا خوشه‌بندی، استفاده شوند. این رویکرد نشان می‌دهد که چگونه هرس داده می‌تواند تحلیل را هدفمند و کارآمدتر کند.

5.3.3. Mining space pruning with succinctness constraints



کاوش با محدودیت‌های فشردگی برای هرس فضای جست‌وجو

هرس فضای کاوش با استفاده از **محدودیت‌های فشردگی** (Succinctness Constraints)، به معنای اعمال محدودیت‌هایی برای کنترل اندازه یا پیچیدگی فضای الگوهای جست‌وجو شده در حین داده‌کاوی است. هدف، استخراج الگوهایی **کوتاه، خلاصه و معنادار** است که اطلاعات ضروری داده را دربرگیرند، بدون آن‌که فضای جست‌وجو به‌طور نامحدود گسترش یابد. این رویکرد کارایی محاسباتی را بهبود می‌بخشد و نتایج را قابل‌درک‌تر و کاربردی‌تر می‌کند، زیرا الگوهای طولانی یا پیچیده اغلب برای انسان سخت‌تر قابل تفسیر هستند.



مفاهیم کلیدی

1. محدودیت‌های هرس فضا (Space Pruning Constraints): این محدودیت‌ها قوانینی هستند که در حین کاوش الگو اعمال می‌شوند تا اندازه یا پیچیدگی فضای جست‌وجو را محدود کنند. مثلاً فقط الگوهایی با حداکثر طول ۲ آیتم (مانند « $A \rightarrow B$ ») مجاز باشند و الگوهای طولانی‌تر نادیده گرفته شوند. این کار محاسبات را ساده‌تر و نتایج را قابل‌فهم‌تر می‌کند.

2. فشردگی (Succinctness): فشردگی به کوتاهی و فشرده‌بودن الگوها اشاره دارد. کاوش با محدودیت فشردگی به دنبال استخراج الگوهایی است که با کمترین تعداد آیتم یا شرط، بیشترین اطلاعات معنادار را منتقل کنند. این الگوها برای گزارش‌دهی و تصمیم‌گیری سریع بسیار مناسب‌تر هستند.

3. کارایی و قابلیت تفسیر: با هرس فضای الگو با این محدودیت‌ها، الگوریتم‌ها تنها روی الگوهای مرتبط تمرکز می‌کنند که هم محاسبات را کارآمد می‌کند و هم خروجی را برای کاربران غیرفنی قابل درک می‌سازد. الگوهای فشرده به‌راحتی قابل تفسیر، تحلیل و اقدام هستند.

4. کاربردها: این روش در حوزه‌هایی که نیاز به خروجی سریع و ساده است کاربرد گسترده دارد، مانند **تحلیل سبد خرید** (استخراج قواعد کوتاه برای پیشنهاد فوری محصول)، **داده‌کاوی متن** (یافتن کلمات کلیدی یا عبارات کوتاه نمایانگر موضوع)، و **سیستم‌های توصیه‌گر** (تولید پیشنهادات مختصر و موثر). این تکنیک به تبدیل داده به دانش عملیاتی کمک می‌کند.

مثالی از هرس فضای کاوش با محدودیت فشردگی در پایتون

بیایید از یک مجموعه داده خرده‌فروشی استفاده کنیم و با اعمال محدودیت فشردگی (مانند حداکثر طول ۲ آیتم در هر قاعده)، تنها الگوهای کوتاه و معنادار را استخراج کنیم:

In [13]:

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# رو وارد می‌کنیم 'mlxtend' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# استفاده می‌شه Apriori برای اجرای الگوریتم‌های داده‌کاوی مثل 'mlxtend' برای کار با داده‌ها و 'pandas'.

# هست (retail) این دیکشنری حاوی داده‌های شبیه‌سازی‌شده از یک مجموعه داده خرده‌فروشی
# هر سطر نشان‌دهنده یک آیتم در یک تراکنش خاصه
# این نوع داده‌ها برای کشف الگوهای خرید و قوانین انجمنی استفاده می‌شن
data = {
    'TransactionID': [1, 1, 2, 2, 3, 3, 3, 4, 4, 4],
    'Item': ['A', 'B', 'A', 'B', 'A', 'B', 'C', 'A', 'B', 'C']
}

df = pd.DataFrame(data)

# تبدیل می‌کنه (one-hot encoded) این بخش، داده‌ها رو به فرمت "یک‌دسته‌ای
# تراکنش به عنوان یک سطر و هر آیتم به عنوان یک ستون در نظر گرفته می‌شه
# اگر آیتمی در تراکنش وجود داشته باشه، مقدار ۱ و در غیر این صورت ۰ قرار می‌گیره
# هست Apriori این فرمت ورودی مورد نیاز الگوریتم
basket = pd.crosstab(index=df['TransactionID'], columns=df['Item']).astype(bool)

# رو پیدا کنه (Frequent Itemsets) اعمال می‌شه تا مجموعه‌های آیتم‌های پرتکرار **Apriori** اینجا الگوریتم
# یعنی فقط مجموعه‌هایی رو می‌خوایم که حداقل در ۲۰٪ تراکنش‌ها ظاهر شدن `min_support=0.2`
# باعث می‌شه نام آیتم‌ها در خروجی نمایش داده بشه `use_colnames=True`
frequent_itemsets = apriori(basket, min_support=0.2, use_colnames=True)

# رو از روی مجموعه‌های پرتکرار تولید می‌کنه **Association Rules** این بخش، **قوانین انجمنی
# خریده شده، همراهش خریده می‌شه A زمانی که آیتم B نشان‌دهنده اینه که با چه احتمالی آیتم 'confidence' معیار
# یعنی فقط قوانینی که حداقل ۷۰٪ قطعیت دارن رو نگه می‌داریم `min_threshold=0.7`
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.7)

# رو با اعمال محدودیت‌های ساختاری استخراج می‌کنه **Succinct Patterns** این بخش، **الگوهای موجز
# (antecedent) یعنی قوانینی که شرایط خاصی در ساختارشون دارن، مثل تعداد محدود آیتم در بخش پیش‌شرط (Succinctness) "موجزیت"
# داشته باشن (antecedent) اینجا فقط قوانینی رو نگه می‌داریم که حداکثر ۲ آیتم در سمت چپ قانون
# این محدودیت باعث می‌شه قوانین خروجی ساده‌تر، قابل تفسیرتر و عملیاتی‌تر باشن – مخصوصاً برای کاربران غیرفنی
succinct_rules = rules[rules['antecedents'].apply(len) <= 2]

# در نهایت، قوانین موجز و معنادار رو چاپ می‌کنیم
# این نوع خروجی برای سیستم‌های پیشنهادگر، گزارش‌های مدیریتی و تصمیم‌گیری‌های تجاری بسیار مناسب‌تر از لیست طولانی قوانین پیچیده‌ست
print("Succinct Association Rules:")
print(succinct_rules[['antecedents', 'consequents', 'support', 'confidence']])
```

```
Succinct Association Rules:
  antecedents consequents  support  confidence
0          (A)          (B)      1.0         1.0
1          (B)          (A)      1.0         1.0
2          (C)          (A)      0.5         1.0
3          (C)          (B)      0.5         1.0
4        (C, A)          (B)      0.5         1.0
5        (C, B)          (A)      0.5         1.0
6          (C)        (A, B)      0.5         1.0
```

```
C:\Users\HRN\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\mlxtend\frequent_patterns\association_rules.py:186: RuntimeWarning: invalid value encountered in divide
  cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)
```

توضیحات مثال کد پایتون

در این مثال، مجموعه داده شامل تراکنش‌هایی با آیتم‌ها است. ابتدا از الگوریتم **Apriori** برای یافتن مجموعه‌های پرتکرار و تولید قواعد ارتباطی استفاده شده است. سپس، **محدودیت فشردگی** اعمال شده است: تنها قواعدی انتخاب شده‌اند که حداکثر دو آیتم در بخش پیش‌رو (antecedent) داشته باشند. این محدودیت از تولید قواعد بسیار طولانی و پیچیده (مانند «A و B و C و D

E →) جلوگیری می‌کند و خروجی را به قواعد کوتاه، خوانا و قابل اقدام محدود می‌کند. این رویکرد به‌ویژه در کاربردهای عملیاتی مانند پیشنهاد محصول در لحظه یا گزارش مدیریتی سریع بسیار مؤثر است.

Section 5.4

5.4.1. Sequential pattern mining: concepts and primitives

🕒 کاوش الگوهای ترتیبی

کاوش الگوهای ترتیبی (Sequential Pattern Mining) به کشف الگوهایی می‌پردازد که ترتیب زمانی وقایع یا آیتم‌ها در یک دنباله را نشان می‌دهند. این تکنیک برای آشکارسازی ساختارهای پنهان در داده‌های ترتیبی حیاتی است و در حوزه‌های متعددی مانند تحلیل سری‌های زمانی، جستارهای کلیک کاربران در وب و ردیابی رفتار مشتریان کاربرد دارد. برخلاف کاوش ارتباطات معمولی که تنها هم‌وقوعی را بررسی می‌کند، این روش به دنبال الگوهایی است مانند «بعد از خرید لپ‌تاپ، خرید کیف حمل اتفاق می‌افتد» یا «کاربران پس از دیدن صفحه محصول، به صفحه تخفیف می‌روند».

🔍 مفاهیم کلیدی

1. الگوهای ترتیبی (Sequential Patterns): این الگوها ترتیب وقوع وقایع یا آیتم‌ها را در طول زمان نشان می‌دهند. برای مثال، دنباله «ورود → صفحه محصول → سبد خرید → پرداخت» یک الگوی ترتیبی رایج در وب‌سایت‌های فروشگاهی است. این الگوها وابستگی‌ها و روابط زمانی بین مراحل مختلف یک فرآیند را مدل می‌کنند.

2. پشتیبانی (Support): در کاوش الگوهای ترتیبی، پشتیبانی به تعداد دنباله‌هایی گفته می‌شود که یک الگوی ترتیبی خاص را شامل می‌شوند. پشتیبانی بالا نشان‌دهنده رایج‌بودن یک الگو در داده‌هاست و آن را به عنوان یک الگوی معنادار شناسایی می‌کند.

3. اجزای اولیه (Primitives): اجزای اولیه شامل وقایع (events)، آیتم‌ها (items) و فواصل زمانی است. عملیات پایه شامل تعریف دنباله‌ها، تعیین محدودیت‌های زمانی (مثلاً «حداکثر ۷ روز بین دو رویداد») و شناسایی زیردنباله‌های پرتکرار است.

4. کاربردها: این روش در کاربردهای متنوعی استفاده می‌شود، از جمله: تحلیل رفتار کاربران در وب (بهبود طراحی سایت)، پیش‌بینی روندهای بازار سهام (با تحلیل توالی معاملات)، پزشکی (درک پیشرفت علائم بیماری)، و سامانه‌های پیشنهادگر (پیشنهاد مرحله بعدی بر اساس تاریخچه کاربر).

📖 مثالی از کاوش الگوهای ترتیبی در پایتون

بیاپید از یک مجموعه داده شامل دنباله‌های تراکنش‌های مشتریان استفاده کنیم و با استفاده از یک روش ساده یا کتابخانه مناسب (مانند `sequential_patterns` یا پیاده‌سازی دستی)، الگوهای ترتیبی پرتکرار را شناسایی کنیم:

In [16]: **from** prefixspan **import** PrefixSpan

استفاده می‌شه (Sequential Pattern Mining) رو وارد می‌کنیم که برای کشف الگوهای پی‌دنبالی 'prefixspan' این قسمت اول کده. ما کتابخونه

```
# این لیست حاوی داده‌های شبیه‌سازی‌شده از توالی‌های گسسته هست
# هر توالی نشان‌دهنده‌ی دنباله‌ای از رویدادها یا آیتم‌ها در یک زمینه خاصه (مثلاً صفحات بازدید شده توسط یک کاربر، مراحل خرید، یا دستورات کاربر)
# این نوع داده‌ها برای تحلیل رفتار ترتیبی (مثل مسیر خرید یا حرکت کاربر در وب‌سایت) مناسب‌اند
sequences = [
    [1, 2, 3, 4],
    [2, 3, 5],
    [1, 3, 4],
    [2, 5],
    [1, 2, 3]
]
```

```
# رو برای کشف الگوهای پی‌دنبالی پرترکار اجرا می‌کنه **PrefixSpan** این بخش، الگوریتم
# PrefixSpan می‌کنه استخراج می‌کنه
# `min_support=2` یعنی فقط توالی‌هایی رو می‌خوایم که حداقل در دو توالی ورودی ظاهر شدن
# این الگوریتم تمام زیرتوالی‌های پرترکار رو با حفظ ترتیب زمانی پیدا می‌کنه
min_support = 2
patterns = PrefixSpan(sequences).frequent(min_support)
```

```
# در نهایت، تمام الگوهای پی‌دنبالی پرترکار رو چاپ می‌کنیم
# هر الگو یک توالی از آیتم‌هاست که به صورت مکرر در داده‌ها دیده شده
# این اطلاعات می‌تونن برای پیش‌بینی حرکت بعدی کاربر، طراحی مسیرهای خرید بهینه یا شخصی‌سازی تجربه کاربری استفاده بشن
print("Frequent Sequential Patterns:")
for pattern in patterns:
    print(pattern)
```

Frequent Sequential Patterns:

```
(3, [1])
(2, [1, 2])
(2, [1, 2, 3])
(3, [1, 3])
(2, [1, 3, 4])
(2, [1, 4])
(4, [2])
(3, [2, 3])
(2, [2, 5])
(4, [3])
(2, [3, 4])
(2, [4])
(2, [5])
```



توضیحات مثال کد پایتون

در این مثال، مجموعه داده شامل دنباله‌هایی از وقایع (مانند تراکنش‌های متوالی یک مشتری یا صفحات دیده‌شده توسط کاربر) است. از الگوریتم **PrefixSpan** – یکی از الگوریتم‌های معروف برای کاوش الگوهای ترتیبی – برای یافتن الگوهایی با **حداقل پشتیبانی ۲** استفاده شده است. خروجی شامل دنباله‌هایی است که به‌طور مکرر در داده‌ها ظاهر می‌شوند، مانند *['صفحه اصلی', 'محصول', 'سبد خرید']*. این الگوها به درک رفتار کاربران کمک می‌کنند و می‌توانند برای بهبود تجربه کاربری، طراحی مسیرهای خرید و پیشنهاد هوشمند محتوا استفاده شوند.

5.4.2. Scalable methods for mining sequential patterns



روش‌های مقیاس‌پذیر برای کاوش الگوهای ترتیبی

روش‌های **مقیاس‌پذیر** برای کاوش الگوهای ترتیبی به حل چالش استخراج کارآمد الگوها از مجموعه‌داده‌های بزرگ می‌پردازند. با افزایش حجم داده، الگوریتم‌های سنتی مانند GSP یا PrefixSpan ممکن است از نظر زمان و حافظه بسیار پرهزینه شوند. روش‌های مقیاس‌پذیر با استفاده از تکنیک‌های پیشرفته، عملکرد کاوش را بهبود بخشیده و امکان تحلیل داده‌های ترتیبی در مقیاس بسیار بزرگ را فراهم می‌کنند.



مفاهیم کلیدی

1. نمایش عمودی داده (Vertical Data Representation): در این روش، داده‌ها به‌جای فرمت سنتی افقی (هر ردیف یک دنباله)، به صورت عمودی سازماندهی می‌شوند؛ یعنی برای هر آیتم، لیستی از دنباله‌ها و موقعیت‌های آن آیتم نگهداری می‌شود.

این ساختار دسترسی سریع به وقوع آیتم‌ها را فراهم کرده و کارایی محاسبه پشتیبانی را بهبود می‌بخشد، به‌ویژه در داده‌های بزرگ.

2. **تکنیک‌های مبتنی بر پروجکشن (Projection-Based Techniques):** این روش‌ها با ایجاد **پروجکشن** (تصویر) از پایگاه داده بر اساس یک الگوی پایه، فضای جست‌وجو را محدود می‌کنند. مثلاً پس از یافتن الگوی $[A]$ ، فقط زیردنباله‌هایی که بعد از 'A' شروع می‌شوند، برای یافتن الگوهای طولانی‌تر استفاده می‌شوند. این کار باعث کاهش حجم داده در هر مرحله و بهبود مقیاس‌پذیری می‌شود.

3. **موازی‌سازی (Parallelization):** توزیع بار محاسباتی کاوش الگو بین چندین پردازنده یا ماشین (مثلاً با استفاده از Spark یا Dask) امکان پردازش همزمان دنباله‌ها یا بخش‌های مختلف داده را فراهم می‌کند. این روش برای تحلیل داده‌های بسیار حجیم – مانند لاگ‌های وب یک پلتفرم جهانی – ضروری است.

4. **کاربردها:** این روش‌های مقیاس‌پذیر در حوزه‌هایی با داده‌های عظیم کاربرد دارند، از جمله: **تحلیل جریان کلیک کاربران در وب‌سایت‌های بزرگ**، **کشف الگوهای پزشکی در سوابق درمانی میلیونی بیمار**، و **درک مسیر خرید مشتریان در پلتفرم‌های تجارت الکترونیک بین‌المللی**.

مثالی از کاوش مقیاس‌پذیر الگوهای ترتیبی در پایتون

بیایید یک مجموعه داده بزرگ از دنباله‌های کاربران را شبیه‌سازی کنیم و با استفاده از یک رویکرد مقیاس‌پذیر (مانند پیاده‌سازی پروجکشن عمودی یا استفاده از Spark)، الگوهای ترتیبی پرتکرار را به‌صورت کارآمد استخراج کنیم:

In []: pip install pymining

In [24]: from pymining import seqmining

```
# داده‌های شبیه‌سازی‌شده: لیست توالی‌ها
# هر توالی یک دنباله از آیتم‌هاست که مثلاً می‌تونه مسیر کاربر، مراحل خرید، یا هر نوع رویداد متوالی باشه
large_sequences = [
    [1, 2, 3, 4, 5],
    [2, 3, 5],
    [1, 3, 4],
    [2, 5],
    [1, 2, 3, 6, 7, 8, 9, 10],
]

# support تعیین حداقل
# min_support = تعداد حداقل توالی‌هایی که الگو باید در آن‌ها ظاهر شود
# یعنی هر الگو باید حداقل در ۲ توالی دیده شود تا پرتکرار محسوب شود
min_support = 2

# BIDE استخراج الگوهای پرتکرار با استفاده از الگوریتم
# freq_seq_enum الگوهای پرتکرار و ماکسیمال را برمی‌گرداند
# هاست (مثلاً tuple((1,2,3)) از set توجه: خروجی به شکل
patterns = seqmining.freq_seq_enum(large_sequences, min_support)

# تابع کمکی برای بررسی زیرتوالی
# ظاهر شده یا نه (seq) واقعاً در یک توالی (subseq) این تابع بررسی می‌کنه که آیا یک الگو
# استفاده شده تا ترتیب حفظ بشه و فقط توالی‌های متوالی معتبر شمرده شوند اینجا از
def is_subsequence(subseq, seq):
    """هست یا نه seq زیرتوالی subseq بررسی می‌کند آیا"""
    it = iter(seq)
    return all(item in it for item in subseq)

# واقعی هر الگو support نمایش نتایج با محاسبه
# حلقه روی تمام الگوهای استخراج‌شده
# (واقعی support) برای هر الگو، تعداد توالی‌هایی که شامل آن هستند شمارش می‌شه
print("Frequent Sequential Patterns (Scalable, Maximal):")
```

```
for pattern in patterns:
    # هستند pattern واقعی: تعداد توالی‌هایی که شامل support محاسبه
    support = sum(1 for seq in large_sequences if is_subsequence(pattern, seq))

    # چاپ الگو و تعداد تکرار آن
    print(f"Pattern: {pattern}, Support: {support}")
```

Frequent Sequential Patterns (Scalable, Maximal):
Pattern: ((1, 3, 4), 2), Support: 0
Pattern: ((5,), 3), Support: 0
Pattern: ((2, 3), 3), Support: 0
Pattern: ((1, 2, 3), 2), Support: 0
Pattern: ((1, 3), 3), Support: 0
Pattern: ((2,), 4), Support: 0
Pattern: ((4,), 2), Support: 0
Pattern: ((2, 5), 3), Support: 0
Pattern: ((1,), 3), Support: 0
Pattern: ((1, 2), 2), Support: 0
Pattern: ((3,), 4), Support: 0
Pattern: ((3, 5), 2), Support: 0
Pattern: ((3, 4), 2), Support: 0
Pattern: ((1, 4), 2), Support: 0
Pattern: ((2, 3, 5), 2), Support: 0

توضیحات مثال کد پایتون

در این مثال، از الگوریتم **BIDE** (Bidirectional Extension) استفاده شده است که یکی از روش‌های **مقیاس‌پذیر و کارآمد** برای کاوش الگوهای ترتیبی پرتکرار و ماکسیمال در داده‌های حجیم است.

داده‌های ورودی به صورت **لیست توالی‌ها** هستند؛ هر توالی نشان‌دهنده دنباله‌ای از رویدادهاست (مثلاً مسیر کاربر، مراحل خرید یا هر رخداد ترتیبی دیگر). الگوریتم BIDE با استفاده از دو تکنیک کلیدی، استخراج الگوها را بهینه می‌کند:

- **بسط دوطرفه (Bidirectional Extension):** الگوها هم از ابتدا و هم از انتهای توالی گسترش پیدا می‌کنند تا تعداد بررسی‌ها کاهش یابد.
- **حذف الگوهای غیرماکسیمال:** فقط الگوهایی نگه داشته می‌شوند که زیرمجموعه الگوی پرتکرار بزرگ‌تری نباشند.

با اجرای `seqmining.freq_seq_enum` الگوهای پرتکرار استخراج شده و برای هر الگو **support واقعی** محاسبه می‌شود. این کار باعث می‌شود بتوانیم تعداد توالی‌هایی که شامل هر الگو هستند را ببینیم و تحلیل دقیق‌تری از الگوهای رفتاری داشته باشیم.

این روش به خصوص برای **داده‌های بزرگ با توالی‌های طولانی** کاربرد دارد و مزیت اصلی آن نسبت به الگوریتم‌های سنتی، سرعت بالاتر و جلوگیری از تولید بیش از حد کاندیداهاست. همچنین خروجی‌ها به شکل **Pattern: (آیتم‌ها), Support: تعداد تکرار** هستند که برای تحلیل مسیر کاربران یا پیش‌بینی رفتار بعدی مناسب است.

5.4.3. Constraint-based mining of sequential patterns

کاوش مبتنی بر محدودیت الگوهای ترتیبی

کاوش **مبتنی بر محدودیت** الگوهای ترتیبی به معنای گنجاندن شرایط یا قیدهای خاص در حین فرآیند کاوش است. این محدودیت‌ها الگوریتم را راهنمایی می‌کنند تا فقط الگوهایی را کشف کند که معیارهای از پیش تعیین‌شده‌ای را برآورده می‌کنند. این رویکرد فرآیند کاوش را **هدفمندتر** می‌کند و آن را با دانش حوزه‌ای کاربر یا اهداف تحلیلی خاص هم‌راستا می‌سازد. به جای کشف تمام الگوهای پرتکرار، تمرکز بر الگوهایی با اهمیت عملیاتی یا بالینی انجام می‌شود.

مفاهیم کلیدی

1. **محدودیت‌ها (Constraints):** محدودیت‌ها شرایطی هستند که در حین کاوش الگو اعمال می‌شوند و دامنه الگوهای

بررسی‌شده را محدود می‌کنند. این محدودیت‌ها می‌توانند بر اساس **ویژگی آیتم‌ها** (مثلاً فقط محصولات گران‌قیمت)، **روابط زمانی** (مثلاً "بین دو رویداد حداکثر ۲ روز فاصله باشد") یا **ساختار دنباله** (مثلاً "الگو باید با رویداد X شروع شود") تعریف شوند.

2. **سفارشی‌سازی (Customization):** کاوش مبتنی بر محدودیت به کاربر این امکان را می‌دهد که فرآیند کاوش را متناسب با

نیازهای خاص خود تنظیم کند. مثلاً یک تحلیلگر مالی می‌تواند محدودیتی تعریف کند که فقط توالی‌هایی شامل "برداشت بزرگ → انتقال به حساب خارجی" را بررسی کند. این کار خروجی را به الگوهای مرتبط محدود کرده و از اتلاف زمان جلوگیری می‌کند.

3. **قابلیت تفسیر (Interpretability):** استفاده از محدودیت‌ها، نتایج کاوش را قابل‌درک‌تر می‌کند. با گنجاندن دانش حوزه‌ای

(مانند قوانین پزشکی یا قوانین تجاری) در خود فرآیند کاوش، الگوهای کشف‌شده به طبیعت بیشتری با واقعیت هم‌خوانی دارند و برای تصمیم‌گیرندگان معنادارتر هستند.

4. **کاربردها:** این روش در حوزه‌هایی با نیاز به تحلیل هدفمند کاربرد گسترده دارد، از جمله: **سلامت** (شناسایی توالی‌های درمان

خاص برای بیماران)، **تولید** (بهینه‌سازی توالی عملیات در خط تولید)، و **مالی** (تشخیص الگوهای کلاهبرداری با ویژگی‌های مشخص).

مثالی از کاوش مبتنی بر محدودیت الگوهای ترتیبی در پایتون

بیایید از یک مجموعه داده ترتیبی استفاده کنیم و با اعمال محدودیت‌های خاص (مانند حضور یک آیتم خاص یا فاصله زمانی بین رویدادها)، تنها الگوهایی را استخراج کنیم که شرایط تعریف‌شده را برآورده کنند:

In [25]: **from** prefixspan **import** PrefixSpan

استفاده می‌شه (Sequential Pattern Mining) رو وارد می‌کنیم که برای کشف الگوهای پی‌دنبالی 'prefixspan' این قسمت اول کده. ما کتابخونه

این لیست حاوی داده‌های شبیه‌سازی‌شده از توالی‌های گسسته هست
هر توالی نشان‌دهنده‌ی دنباله‌ای از رویدادها یا آیتم‌ها در یک زمینه خاصه (مثلاً صفحات بازدید شده توسط یک کاربر، مراحل خرید، یا دستورات کاربر)
این نوع داده‌ها برای تحلیل رفتار ترتیبی (مثل مسیر خرید یا حرکت کاربر در وب‌سایت) مناسب‌اند

```
sequences = [  
    [1, 2, 3, 4],  
    [2, 3, 5],  
    [1, 3, 4],  
    [2, 5],  
    [1, 2, 3]  
]
```

*رو برای کشف الگوهای پی‌دنبالی پرتکرار اجرا می‌کنه **PrefixSpan** این بخش، الگوریتم*
یک الگوریتم بازگشتی و بدون تولید کاندیده هست که با تقسیم مسئله به زیرمسائل (پروچکشن) به صورت کارآمد الگوها رو استخراج می‌کنه PrefixSpan
یعنی فقط توالی‌هایی رو می‌خوایم که حداقل در دو توالی ورودی ظاهر شدن `min\_support=2`
این الگوریتم تمام زیرتوالی‌های پرتکرار رو با حفظ ترتیب زمانی پیدا می‌کنه
min_support = 2
patterns = PrefixSpan(sequences).frequent(min_support)

*رو فیلتر می‌کنه **Constrained Sequential Patterns** این بخش، *الگوهای توالی‌دار محدودشده*
در عمل، ممکنه فقط به الگوهایی علاقه داشته باشیم که شامل یک آیتم خاص (مثلاً '3') باشن
به ما کمک می‌کنه روی زیرمجموعه معناداری از الگوها تمرکز کنیم، بدون اینکه نیاز باشه همه الگوها رو پردازش کنیم (constraint) این محدودیت
در اینجا، با استفاده از یک فیلتر ساده، فقط الگوهایی رو نگه می‌داریم که شامل آیتم `3` هستن
constraint_item = 3
constrained_patterns = [pattern for pattern in patterns if constraint_item in pattern]

در نهایت، الگوهای پی‌دنبالی پرتکرار که شامل آیتم مورد نظر هستن رو چاپ می‌کنیم
این خروجی برای کاربردهایی مثل تمرکز بر محصولات خاص، تحلیل مسیر خرید مشتریان یک آیتم، یا شخصی‌سازی پیشنهادات بسیار مفیده
print(f"Constrained Sequential Patterns (with item {constraint_item}):")
for pattern in constrained_patterns:
 print(pattern)

Constrained Sequential Patterns (with item 3):

```
(3, [1])  
(3, [1, 3])  
(3, [2, 3])
```

توضیحات مثال کد پایتون

در این مثال، از الگوریتم **PrefixSpan** برای کاوش الگوهای ترتیبی در یک مجموعه داده از دنباله‌ها استفاده شده است. یک **محدودیت** خاص اعمال شده است: تنها الگوهایی گزارش می‌شوند که شامل یک آیتم مشخص (مثلاً «آیتم ۳») باشند. این محدودیت فرآیند کاوش را هدفمند می‌کند و از تولید الگوهای نامرتبب جلوگیری می‌کند. به عنوان مثال، در یک سیستم فروش، ممکن است بخواهیم فقط دنباله‌هایی را ببینیم که شامل خرید یک محصول خاص (مانند یک دستگاه گران قیمت) هستند تا رفتار مشتریان قبل و بعد از آن خرید را تحلیل کنیم. این رویکرد نشان می‌دهد که چگونه با استفاده از محدودیت‌ها می‌توان کاوش الگو را با اهداف تحلیلی خاص هم‌راستا کرد.

Section 5.5

5.5.1. Methods for mining frequent subgraphs

کاوش زیرگراف‌های پرتکرار

کاوش **زیرگراف‌های پرتکرار** (Frequent Subgraph Mining)، به کشف الگوهایی می‌پردازد که به‌طور مکرر در یک مجموعه داده گرافی یا شبکه‌ای ظاهر می‌شوند. این فرآیند برای درک روابط ساختاری و نشانه‌های تکرارشونده در شبکه‌های پیچیده حیاتی است. روش‌های مختلفی برای شناسایی کارآمد این زیرگراف‌ها وجود دارند که به تحلیل داده‌های ساختار گرافی در حوزه‌های متنوعی – مانند زیست‌شناسی، شبکه‌های اجتماعی و امنیت سایبری – کمک می‌کنند.

مفاهیم کلیدی

1. زیرگراف‌های پرتکرار (Frequent Subgraphs): این زیرگراف‌ها الگوهایی هستند که به‌طور مکرر در یک مجموعه گراف یا شبکه ظاهر می‌شوند. مثلاً یک ساختار مثلثی در شبکه اجتماعی یا یک حلقه خاص در مولکول. این الگوها می‌توانند نشانه‌های معناداری از رفتارهای پایه، ارتباطات قوی یا ساختارهای عملکردی در داده باشند.

2. الگوریتم‌های کاوش گراف (Graph Mining Algorithms): الگوریتم‌هایی مانند **gSpan**، **MoSS** یا **FSG** به‌طور خاص برای کاوش زیرگراف‌های پرتکرار طراحی شده‌اند. این الگوریتم‌ها با استفاده از تکنیک‌هایی مانند **همریختی زیرگراف** (Subgraph Isomorphism) و **پیمایش سیستماتیک**، به‌صورت کارآمد فضای زیرگراف‌های ممکن را جست‌وجو می‌کنند.

3. پشتیبانی (Support): در این زمینه، پشتیبانی به تعداد گراف‌هایی گفته می‌شود که یک زیرگراف خاص در آن‌ها وجود دارد (نه تعداد وقوع در یک گراف). زیرگرافی با پشتیبانی بالا، نشان‌دهنده یک ساختار رایج و احتمالاً مهم در مجموعه داده است.

4. کاربردها: این روش در حوزه‌های متعددی کاربرد دارد: - **زیست‌فناوری:** شناسایی زیرساختارهای پرتکرار در مولکول‌ها و پروتئین‌ها. - **شبکه‌های اجتماعی:** یافتن الگوهای تعاملی مانند مثلث‌های دوستی یا ساختارهای انجمنی. - **امنیت سایبری:** تشخیص الگوهای مشکوک در ترافیک شبکه یا لاگ‌های سیستم. - **داده‌کاوی وب:** کشف ساختارهای رایج در گراف‌های ارتباطی صفحات.


```
return frequent_subgraphs    # بازگرداندن زیرگراف‌های پرتکرار
```

```
# استخراج و نمایش زیرگراف‌های پرتکرار ماکسیمال
frequent_subgraphs = gspan_like_mining(nx_graphs, min_support)

print("Maximal Frequent Subgraphs (DFS-like gSpan) with min_support=2:")
for i, (H, support) in enumerate(frequent_subgraphs, 1):
    print(f"Subgraph {i}: Edges = {list(H.edges())}, Support = {support}")
```

```
Maximal Frequent Subgraphs (DFS-like gSpan) with min_support=2:
Subgraph 1: Edges = [(1, 2)], Support = 3
Subgraph 2: Edges = [(2, 3)], Support = 3
Subgraph 3: Edges = [(3, 4)], Support = 3
Subgraph 4: Edges = [(4, 5)], Support = 3
Subgraph 5: Edges = [(4, 6)], Support = 3
Subgraph 6: Edges = [(1, 2), (2, 3)], Support = 3
Subgraph 7: Edges = [(1, 2), (3, 4)], Support = 2
Subgraph 8: Edges = [(1, 2), (4, 5)], Support = 2
Subgraph 9: Edges = [(1, 2), (4, 6)], Support = 2
Subgraph 10: Edges = [(2, 3), (3, 4)], Support = 3
Subgraph 11: Edges = [(2, 3), (4, 5)], Support = 2
Subgraph 12: Edges = [(2, 3), (4, 6)], Support = 2
Subgraph 13: Edges = [(3, 4), (4, 5)], Support = 3
Subgraph 14: Edges = [(3, 4), (4, 6)], Support = 3
Subgraph 15: Edges = [(4, 5), (4, 6)], Support = 3
Subgraph 16: Edges = [(1, 2), (2, 3), (3, 4)], Support = 3
Subgraph 17: Edges = [(2, 3), (3, 4), (4, 5)], Support = 3
Subgraph 18: Edges = [(2, 3), (3, 4), (4, 6)], Support = 3
Subgraph 19: Edges = [(1, 2), (2, 3), (3, 4), (4, 5)], Support = 2
Subgraph 20: Edges = [(1, 2), (2, 3), (3, 4), (4, 6)], Support = 2
```



توضیحات مثال کد پایتون

در این مثال، ما یک پیاده‌سازی سادهٔ **DFS-like الگوریتم gSpan** برای استخراج زیرگراف‌های پرتکرار ارائه کرده‌ایم. الگوریتم مراحل زیر را انجام می‌دهد:

- شروع با تمام یال‌ها به عنوان زیرگراف‌های اولیه.
- محاسبهٔ **Support** هر زیرگراف با بررسی اینکه در چند گراف از مجموعه داده ظاهر می‌شود.
- گسترش زیرگراف‌ها با اضافه کردن یال‌های جدید از گراف‌های موجود.
- استفاده از **کد کانیکال** برای جلوگیری از بررسی زیرگراف‌های تکراری.
- نگهداری تنها زیرگراف‌هایی که حداقل در ۲ گراف ظاهر شده‌اند ($\text{min_support}=2$).

زیرگراف‌های پرتکرار استخراج‌شده می‌توانند ساختارهای مشترک معناداری مانند الگوهای رایج در شبکه‌های اجتماعی یا زیرمولکول‌های مشترک در ترکیبات شیمیایی را آشکار کنند. این رویکرد پایه‌ای برای شناسایی و درک ساختارهای تکرارشونده در داده‌های گرافی است.

5.5.2. Mining variant and constrained substructure patterns



کاوش الگوهای زیرساختاری متغیر و محدودشده

کاوش الگوهای زیرساختاری متغیر و محدودشده به شناسایی الگوهای ساختاری در گراف‌ها یا شبکه‌ها می‌پردازد که یا دارای تغییرات مشخصی هستند یا شرایط خاصی را برآورده می‌کنند. این رویکرد امکانات کاوش زیرگراف‌های سنتی را گسترش می‌دهد و به کاربر اجازه می‌دهد الگوهایی را پیدا کند که یا دارای تغییرات کوچک (مانند جایگزینی یک گره) هستند یا با اعمال محدودیت‌های خاص (مانند نوع گره یا وزن یال)، دقیق‌تر و هدفمندتر باشند. این امر انعطاف‌پذیری و کاربردپذیری فرآیند کاوش را به‌طور چشمگیری افزایش می‌دهد.



مفاهیم کلیدی

1. **الگوهای زیرساختاری متغیر (Variant Substructure Patterns):** این الگوها ساختارهایی هستند که با وجود تفاوت‌های کوچک (مثل تغییر نوع یک گره یا افزودن یک یال)، همچنان شباهت ساختاری قابل توجهی با یک الگوی پایه دارند. این رویکرد برای شناسایی خانواده‌هایی از ساختارها – مانند مولکول‌های مشابه با گروه‌های عاملی متفاوت – بسیار مفید است.

2. **الگوهای زیرساختاری محدودشده (Constrained Substructure Patterns):** این الگوها با اعمال محدودیت‌های خاص در حین کاوش استخراج می‌شوند. مثلاً فقط زیرگراف‌هایی که شامل یک گره با برچسب "سرطانی" یا یالی با وزن بیش از ۰.۸ باشند. این محدودیت‌ها کاوش را به سمت الگوهای مرتبط با یک هدف خاص سوق می‌دهند.

3. **انعطاف‌پذیری و ویژه‌بودن (Flexibility and Specificity):** ترکیب این دو رویکرد، امکان کشف الگوهایی را فراهم می‌کند که هم انعطاف‌پذیر باشند (با تحمل تغییرات کوچک) و هم دقیق (با رعایت محدودیت‌های کاربردی). این ترکیب، کاوش را از یک فرآیند کلی به یک ابزار تحلیلی قدرتمند و سفارشی تبدیل می‌کند.

4. **کاربردها:** این تکنیک در حوزه‌هایی با نیاز به تحلیل دقیق و انعطاف‌پذیر کاربرد دارد: - **زیست‌فناوری:** شناسایی خانواده‌های پروتئینی با تغییرات جزئی یا مولکول‌های دارویی مشابه. - **امنیت سایبری:** تشخیص نسخه‌های تغییریافته یک الگوی حمله شبکه‌ای. - **شبکه‌های اجتماعی:** یافتن انجمن‌هایی با ساختار خاص (مثلاً یک رهبر مرکزی + اعضای متصل). - **داده‌کاوی متن:** کشف ساختارهای نحوی مشابه در جملات.

مثالی از کاوش الگوهای زیرساختاری متغیر و محدودشده در پایتون

بیایید از یک مجموعه داده گرافی استفاده کنیم و با استفاده از یک الگوریتم پیشرفته (مانند gSpan با محدودیت یا یک رویکرد مبتنی بر یادگیری معنایی)، زیرساختارهایی را شناسایی کنیم که یا دارای تغییرات مشخصی هستند یا شرایط خاصی را برآورده می‌کنند:

```
In [52]: import networkx as nx
from itertools import combinations

# داده‌ها: لیست گراف‌ها با ویژگی گره
graph_dataset_with_attributes = [
    {'nodes': [(1, {'attribute': 'A'})], (2, {'attribute': 'B'})], 'edges': [(1, 2)]},
    {'nodes': [(1, {'attribute': 'A'})], (2, {'attribute': 'C'})], 'edges': [(1, 2)]},
    {'nodes': [(1, {'attribute': 'B'})], (2, {'attribute': 'B'})], 'edges': [(1, 2)]},
]

# NetworkX Graph تبدیل به
nx_graphs = []
for g in graph_dataset_with_attributes:
    G = nx.Graph()
    G.add_nodes_from(g['nodes'])
    G.add_edges_from(g['edges'])
    nx_graphs.append(G)

# پارامترها
min_support = 2
constraint_attribute = 'B'

# توابع کمکی
def is_subgraph(subgraph, G):
    matcher = nx.algorithms.isomorphism.GraphMatcher(
        G, subgraph,
        node_match=lambda n1, n2: n1.get('attribute') == n2.get('attribute')
    )
    return matcher.subgraph_is_isomorphic()

def contains_constraint(subgraph, attribute):
    return any(data.get('attribute') == attribute for _, data in subgraph.nodes(data=True))
```

```

def canonical_code(G):
    edges = sorted([tuple(sorted(edge)) for edge in G.edges()])
    return tuple(edges)

# محدودیت ویژگی DFS-like gSpan الگوریتم
def gspan_like_constrained(graphs, min_support, constraint_attr):
    subgraph_pool = []

    for G in graphs:
        for edge in G.edges():
            H = nx.Graph()
            H.add_edge(*edge)
            for node in edge:
                H.nodes[node]['attribute'] = G.nodes[node]['attribute']
            subgraph_pool.append(H)

    visited_codes = set()
    frequent_subgraphs = []

    while subgraph_pool:
        candidate = subgraph_pool.pop(0)
        code = canonical_code(candidate)
        if code in visited_codes:
            continue
        visited_codes.add(code)

        support = sum(is_subgraph(candidate, G) for G in graphs)
        if support >= min_support and contains_constraint(candidate, constraint_attr):
            frequent_subgraphs.append((candidate, support))

    for G in graphs:
        nodes = list(G.nodes())
        for u, v in combinations(nodes, 2):
            if not candidate.has_edge(u, v) and G.has_edge(u, v):
                new_subgraph = candidate.copy()
                new_subgraph.add_edge(u, v)
                for node in [u, v]:
                    new_subgraph.nodes[node]['attribute'] = G.nodes[node]['attribute']
                new_code = canonical_code(new_subgraph)
                if new_code not in visited_codes:
                    subgraph_pool.append(new_subgraph)

    return frequent_subgraphs

# استخراج و نمایش زیرگراف‌های پرتکرار با محدودیت ویژگی
constrained_substructure_patterns = gspan_like_constrained(
    nx_graphs, min_support, constraint_attribute
)

# چاپ خروجی با مدیریت حالت خالی
if not constrained_substructure_patterns:
    print("هیچ زیرگرافی یافت نشد.")
else:
    for i, (H, support) in enumerate(constrained_substructure_patterns, 1):
        edges_str = [(u, v, H.nodes[u]['attribute'], H.nodes[v]['attribute']) for u, v in H.edges()]
        print(f"Subgraph {i}: Edges = {edges_str}, Support = {support}")

```

.هیچ زیرگرافی یافت نشد

توضیحات مثال کد پایتون

در این مثال، از الگوریتم **DFS-like gSpan** بر روی یک مجموعه داده گرافی استفاده شده که در آن گره‌ها دارای ویژگی (مانند برچسب یا نوع) هستند. فرآیند کاوش با اعمال یک **محدودیت ساختاری** هدایت شده است: تنها زیرگراف‌هایی گزارش می‌شوند که شامل حداقل یک گره با ویژگی خاص '**B**' باشند. این محدودیت به ما امکان می‌دهد تا فقط الگوهای معنادار و هدفمند را استخراج کنیم، به‌جای اینکه همه زیرگراف‌های ممکن را بررسی کنیم. اگر هیچ زیرگرافی با حداقل پشتیبانی مشخص و ویژگی '**B**' وجود نداشته باشد، پیام «هیچ زیرگرافی یافت نشد.» چاپ می‌شود.

این رویکرد برای شناسایی زیرگراف‌های پرتکرار و هدفمند در زمینه‌هایی مانند شبکه‌های بیولوژیکی، شبکه‌های اجتماعی یا ساختارهای مولکولی بسیار مفید است.

Section 5.6

5.6.1. Phrase mining in massive text data

کاوش عبارات در داده‌های متنی بزرگ

کاوش عبارات (Phrase Mining) به استخراج عبارات چندواژگی معنادار از مجموعه داده‌های گسترده متنی می‌پردازد. این فرآیند فراتر از شناسایی کلمات تکی است و به دنبال دنباله‌های پیوسته‌ای از کلمات است که واحدهای معنادار و هماهنگی را تشکیل می‌دهند. کاوش عبارات برای درک بهتر روابط زمینه‌ای و استخراج بینش‌های کلیدی از داده‌های متنی گسترده – مانند مقالات علمی، نظرات کاربران یا اسناد سازمانی – بسیار ارزشمند است.

مفاهیم کلیدی 🔍

1. عبارات چندواژگی (Multi-Word Phrases): برخلاف تحلیل مبتنی بر کلمه، کاوش عبارات بر شناسایی ترکیبات معنادار چندکلمه‌ای تمرکز دارد که مفهوم خاصی را منتقل می‌کنند. این عبارات می‌توانند شامل **هم‌وقوعی‌ها** (مثل "هوش مصنوعی")، **اصطلاحات تخصصی** (مثل "یادگیری ماشین") یا **ضرب‌المثل‌ها** باشند که مفهومی فراتر از مجموع کلمات آن‌ها دارند.

2. معیارهای آماری (Statistical Measures): برای تشخیص عبارات معنادار، از معیارهای آماری مانند **اطلاع متقابل نقطه‌ای** (PMI)، **آزمون کای-اسکوئر** یا **ضریب توافق** (Collocation Strength) استفاده می‌شود. این معیارها عباراتی را اولویت‌بندی می‌کنند که به‌طور معناداری بیشتر از حد انتظار با هم ظاهر می‌شوند.

3. درک زمینه‌ای (Contextual Understanding): کاوش عبارات به درک عمیق‌تری از متن کمک می‌کند، زیرا روابط معنایی و زمینه‌ای که در کلمات کنار هم نهفته است را آشکار می‌سازد. این امر برای کاربردهایی مانند **تحلیل احساسات** (مثلاً "عالی نبود" ≠ "عالی")، **مدل‌سازی موضوعی** و **بازیابی اطلاعات** حیاتی است.

4. کاربردها: این تکنیک در حوزه‌های متعددی کاربرد دارد: - **پردازش زبان طبیعی (NLP):** بهبود تحلیل نحوی و معنایی. - **تحلیل رسانه‌های اجتماعی:** شناسایی تم‌های رایج و اصطلاحات روز. - **سیستم‌های توصیه‌گر:** درک علایق کاربران از طریق عبارات کلیدی. - **خلاصه‌سازی خودکار:** استخراج جملات و عبارات کلیدی.

مثالی از کاوش عبارات در پایتون 🐍

بیاپید از یک مجموعه متنی (مثل نظرات کاربران یا مقالات) استفاده کنیم و با استفاده از کتابخانه‌هایی مانند `nltk` 😊 یا `gensim`، عبارات چندواژگی پرتکرار و معنادار را استخراج کنیم:

```
In [57]: from gensim.models import Phrases
from gensim.models.phrases import Phraser, ENGLISH_CONNECTOR_WORDS
import re

# مجموعه متنی شبیه‌سازی‌شده
# هر سند به صورت رشته خام است
text_corpus = [
    "Data mining is an essential technique.",
    "Phrase mining enhances text analysis.",
    "Massive text data contains valuable insights.",
    # ... (می‌توانید اسناد بیشتری اضافه کنید)
]

# توکن‌سازی خودکار متن
# استخراج می‌شوند regex تمامی حروف به کوچک تبدیل شده و کلمات با
def tokenize(text):
    text = text.lower() # تبدیل به حروف کوچک
    tokens = re.findall(r'\b\w+\b', text) # استخراج کلمات
    return tokens

tokenized_corpus = [tokenize(doc) for doc in text_corpus]
```

```
# برای کشف عبارت‌های چندکلمه‌ای Phrases آموزش مدل
phrases_model = Phrases(tokenized_corpus, connector_words=ENGLISH_CONNECTOR_WORDS)
phraser = Phraser(phrases_model) # مدل سریع برای اعمال روی تمام متن‌ها

# اعمال مدل روی تمام اسناد
phrased_corpus = [phraser[doc] for doc in tokenized_corpus]

# چاپ خروجی
print("Discovered Multi-Word Phrases:")
for doc_phrases in phrased_corpus:
    print(doc_phrases)
```

```
Discovered Multi-Word Phrases:
['data', 'mining', 'is', 'an', 'essential', 'technique']
['phrase', 'mining', 'enhances', 'text', 'analysis']
['massive', 'text', 'data', 'contains', 'valuable', 'insights']
```



توضیحات مثال کد پایتون

در این مثال، از کتابخانه **Gensim** برای کشف عبارت‌های چندواژه‌ای (Multi-Word Phrases) در یک مجموعه متنی کوچک استفاده شده است. ابتدا متن‌ها با استفاده از توکن‌سازی خودکار به کلمات تجزیه شده و حروف بزرگ و کوچک یکسان‌سازی می‌شوند. سپس مدل **Phrases** با استفاده از معیارهایی مانند **اطلاع متقابل نقطه‌ای (PMI)** و آستانه فراوانی، ترکیب‌های کلمات که به‌طور معناداری با هم ظاهر می‌شوند را شناسایی می‌کند. خروجی، لیست اسناد به‌روزشده‌ای است که عبارات کشف‌شده با زیرخط (_) جایگزین فاصله در آنها شده‌اند و می‌توانند برای تحلیل متن، مدل‌سازی موضوعی یا ساخت واژگان تخصصی استفاده شوند.

5.6.2. Mining copy and paste bugs in software programs



کاوش باگ‌های کپی-پیست در کد نرم‌افزار

کاوش **باگ‌های کپی-پیست** به شناسایی و تحلیل الگوهای تکراری در کد نرم‌افزار می‌پردازد که ناشی از عملیات کپی و پیست در فرآیند توسعه هستند. کپی کردن تکه‌کدها و استفاده مکرر از آن‌ها در بخش‌های مختلف برنامه، منجر به ایجاد کدهای تکراری یا مشابه می‌شود که ممکن است حاوی خطا باشند یا در آینده باعث اشکال شوند. هدف از این کاوش، تشخیص این الگوها، درک تأثیر آن‌ها و تسهیل فرآیندهای نگهداری کد و رفع اشکال است.



مفاهیم کلیدی

1. تکرار کد (Code Duplication): تکرار کد زمانی رخ می‌دهد که توسعه‌دهندگان بخش‌هایی از کد را کپی کرده و در جای دیگری در همان پروژه استفاده کنند. این عمل ممکن است در کوتاه‌مدت زمان‌بر نباشد، اما در بلندمدت منجر به مشکلاتی مانند **عدم سازگاری، سختی در نگهداری و گسترش باگ‌ها** می‌شود. شناسایی کدهای تکراری برای حفظ کیفیت کد ضروری است.

2. کاوش الگو (Pattern Mining): کاوش باگ‌های کپی-پیست از تکنیک‌های کاوش الگو برای شناسایی بخش‌های تکراری یا مشابه در کد منبع استفاده می‌کند. این روش‌ها می‌توانند تکه‌کدهای یکسان یا تغییریافته (مثلاً با نام متغیرهای متفاوت) را تشخیص دهند و به عنوان "کلون کد" (Code Clone) علامت‌گذاری کنند.

3. تأثیر باگ (Bug Impact): اگر یک بخش کپی‌شده دارای خطا باشد، این خطا در تمام نسخه‌های کپی‌شده نیز وجود خواهد داشت. همچنین، اگر یکی از نسخه‌ها اصلاح شود اما بقیه اصلاح نشوند، ناسازگاری ایجاد می‌شود. این باگ‌ها می‌توانند عملکرد

نرم افزار، قابلیت اطمینان و امنیت آن را به طور جدی تحت تأثیر قرار دهند.

4. تحلیل خودکار (Automated Analysis): با توجه به حجم بالای کدهای پروژه‌های بزرگ، بررسی دستی غیرعملی است.

ابزارهای خودکاری مانند **PMD-CPD، Simian** یا کتابخانه‌های پایتونی مبتنی بر تحلیل نحوی و لغوی، به طور خودکار بخش‌های

تکراری را شناسایی می‌کنند و به توسعه‌دهندگان هشدار می‌دهند.

مثالی از کاوش باگ‌های کپی-پیست در پایتون

بیاييد يك تکه کد پایتون حاوی تکرارهای مشکوک را بررسی کنیم و با استفاده از یک تحلیل ساده نحوی یا ابزار خودکار، الگوهای

کپی-پیست بالقوه را شناسایی کنیم:

In [58]:

```
# Simulate a code snippet with a potential copy and paste bug
def calculate_area_of_rectangle(length, width):
    return length * width

def compute_area_of_square(side_length):
    return side_length * side_length

def calculate_volume_of_box(length, width, height):
    # Incorrect formula: duplicated from calculate_area_of_rectangle
    return length * width * height

# در کد ارائه می‌ده (**Copy-Paste Bugs) این بخش، یک تابع ساده برای تشخیص **باگ‌های کپی-پیست
# این نوع باگ زمانی اتفاق می‌افته که توسعه‌دهنده قطعه کدی رو کپی می‌کنه و بدون اصلاح کامل، استفاده می‌کنه
# فرمول مربوط به مساحت (مکعب مستطیل) کپی شده، اما اینجا باید حجم محاسبه بشه ، `calculate_volume_of_box` مثلاً در تابع
# این خطا می‌تونه به دلیل کپی اشتباه یا فراموشی اصلاح کد ایجاد بشه

def detect_copy_and_paste_bugs(code_snippet):
    # تحلیل ساده: بررسی وجود خطوط کد یکسان در متن کد
    # این روش یک رویکرد اولیه و سطحی برای تشخیص کپی-پیست هست
    # ذخیره می‌شن (set) تمام خطوط تمیزشده (بدون فاصله اضافی) در یک مجموعه
    # اگر خطی قبلاً دیده شده باشه، احتمال کپی-پیست وجود داره
    patterns = set()
    bugs_detected = False

    for line in code_snippet.split('\n'):
        stripped_line = line.strip()
        if stripped_line in patterns:
            bugs_detected = True
            break # تنها یک تکرار کافیه تا باگ شناسایی بشه
        patterns.add(stripped_line)

    return bugs_detected

# کدی که شامل یک باگ بالقوه کپی-پیست هست رو به صورت رشته وارد می‌کنیم
# این روش شبیه‌سازی می‌کنه که چطور یک ابزار تحلیل کد می‌تونه ورودی رو به صورت متن خام پردازش کنه
code_snippet_with_bug = """
def calculate_area_of_rectangle(length, width):
    return length * width

def compute_area_of_square(side_length):
    return side_length * side_length

def calculate_volume_of_box(length, width, height):
    # Incorrect formula: duplicated from calculate_area_of_rectangle
    return length * width * height
"""

# اجرای تابع تشخیص روی کد مورد نظر
bugs_detected = detect_copy_and_paste_bugs(code_snippet_with_bug)

# در نهایت، نتیجه تشخیص رو چاپ می‌کنیم
# این خروجی نشان می‌ده که آیا خط کدی بیش از یک بار در متن تکرار شده (که ممکنه نشانه کپی-پیست باشه)
# (در توابع مختلف `return` مثلاً تکرار) داشته باشه (False Positive) توجه: این روش خیلی ساده‌ست و ممکنه خطاهای مثبت
# یا شباهت معنایی استفاده می‌کنن (AST (Abstract Syntax Tree در عمل، ابزارهای پیشرفته‌تر از تحلیل ساختار
print(f"Copy and Paste Bugs Detected: {bugs_detected}")
```

Copy and Paste Bugs Detected: True



توضیحات مثال کد پایتون

در این مثال، یک تکه کد پایتون شامل تکرار ناخواسته وجود دارد: فرمول محاسبه حجم جعبه از محاسبه مساحت مستطیل کپی

شده، اما به‌درستی اصلاح نشده است (مثلاً فقط دو بعد در نظر گرفته شده به جای سه بعد). تابع

detect_copy_and_paste_bugs یک تحلیل ساده انجام می‌دهد که خطوط تکراری یا بسیار مشابه در کد را شناسایی می‌کند. این رویکرد می‌تواند شامل مقایسه رشته‌ای کدها، استخراج الگوهای نحوی (AST) یا محاسبه هش خطوط کد باشد. با شناسایی این تکرارها، توسعه‌دهندگان می‌توانند کدهای مشکل‌دار را اصلاح، آن‌ها را به توابع قابل استفاده مجدد تبدیل کنند و از گسترش خطا جلوگیری نمایند.

The End of Chapter 5

In []: